Lec06：创建型设计模式

Creational Design Patterns

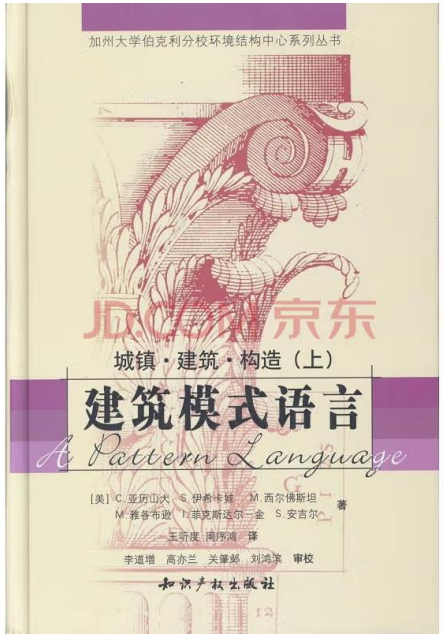
课程编号：22WHSE22006（2023版培养计划）
授课对象：23 级软件工程、服务科学与工程专业
主讲教师：辛国栋
Email：xinguodong@163.com



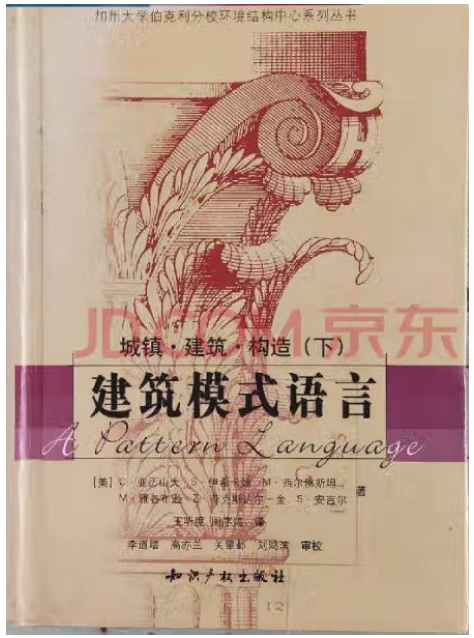
QQ 群：软件构造2025
群　号：311528424

引言：软件设计模式的产生背景

- 设计模式（Design Pattern，业界简称 DP）起源于建筑设计。



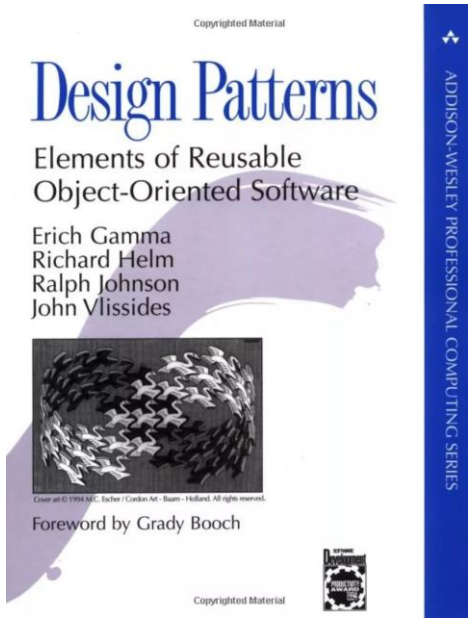


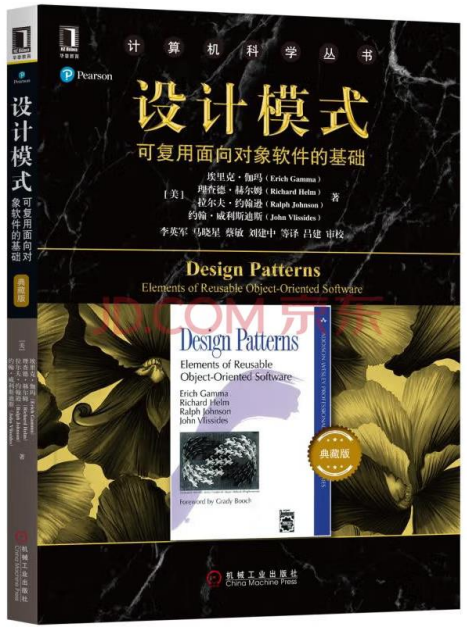


  - 1977年美国建筑大师 克里斯托夫·亚历山大（Christopher Alexander）在他的著作《建筑模式语言：城镇、建筑、构造》中描述了一些常见的建筑设计问题，并提出了 253 种关于对城镇、邻里、住宅、花园和房间等进行设计的基本模式。

引言：软件设计模式的产生背景

- 1995年，Erich Gamma、Richard Helm、Ralph Johnson和 John Vlissides 合作出版了《设计模式：可复用面向对象软件的基础》一书，此书收录了 23 种设计模式，这是设计模式领域里程碑的事件，导致了软件设计模式的突破。

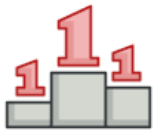




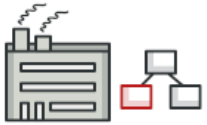


  - 这 4 位作者在软件开发领域里称为“四人帮”（GoF，Gang of Four）

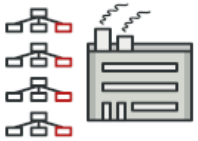
引言：设计模式总览



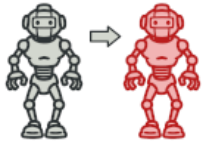
单例
Singleton



工厂方法
Factory Method



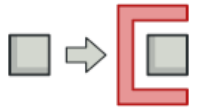
抽象方法
Abstract Factory



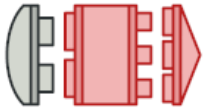
原型
Prototype



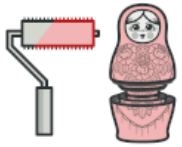
建造者
Builder



代理
Proxy



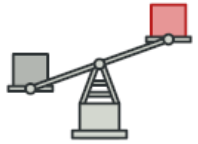
适配器
Adapter



装饰
Decorator



桥接
Bridge



享元
Flyweight



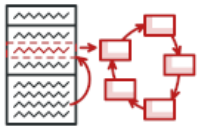
组合
Composite



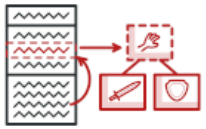
外观
Facade



观察者
Observer



状态
State



策略
Strategy



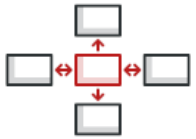
模版方法
Template Method



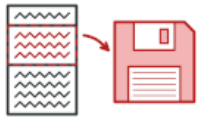
访问者
Visitor



责任链
Chain of Responsibility



中介
Mediator



备忘录
Memento



迭代器
Iterator



命令
Command

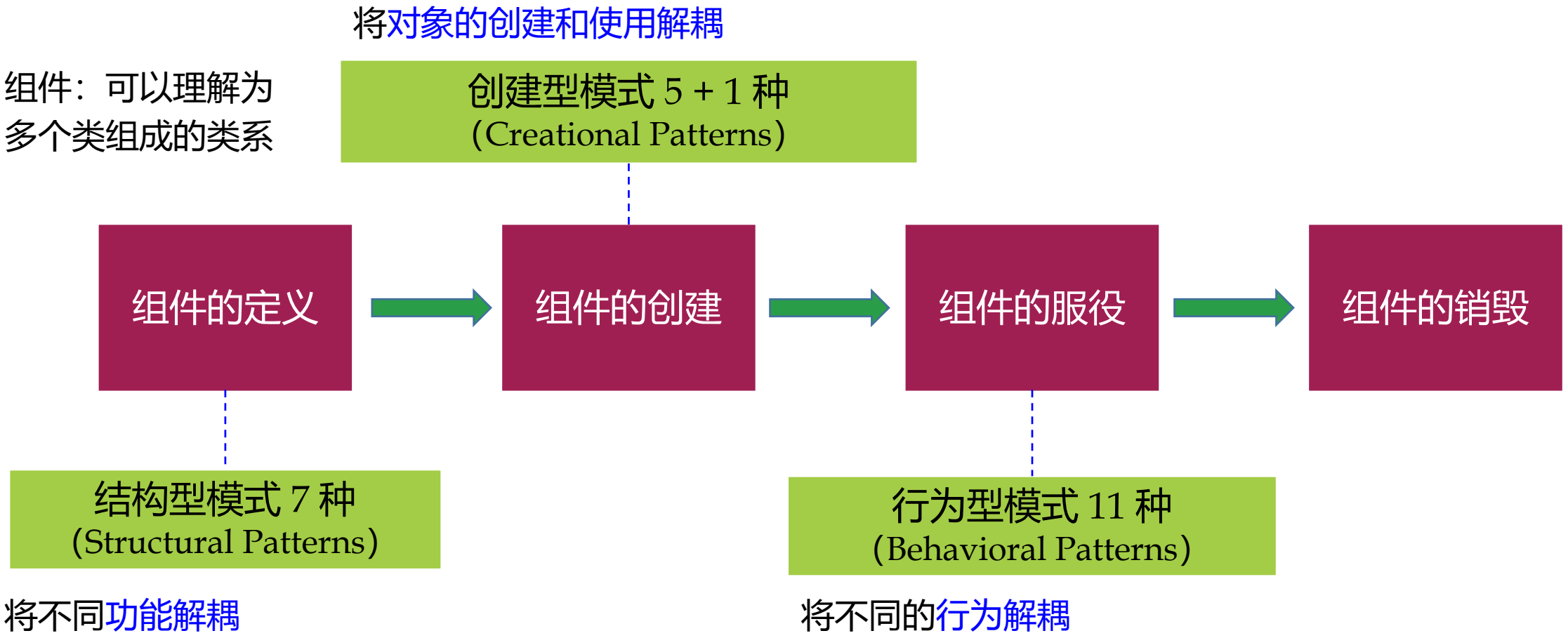


解释器
Interpreter

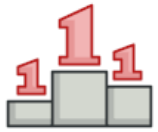


编外：简单工厂
Simple Factory

引言：对 23+1 种模式按组件的生命周期进行分类



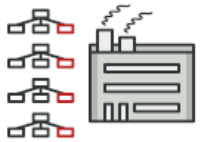
创建型模式(5+1 种)
Creational Patterns



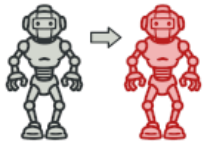
单例
Singleton



工厂方法
Factory Method



抽象工厂
Abstract Factory



原型
Prototype



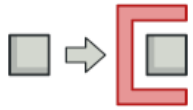
建造者
Builder



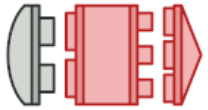
编外：简单工厂
Simple Factory

对象产生与使用解耦

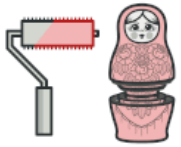
结构型模式(7 种)
Structural Patterns



代理
Proxy



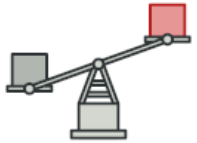
适配器
Adapter



装饰
Decorator



桥接
Bridge



享元
Flyweight



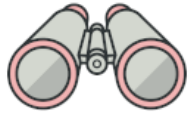
组合
Composite



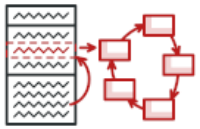
外观
Facade

功能解耦

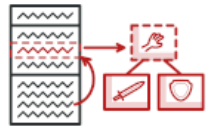
行为型模式 (11 种)
Behavioral Patterns



观察者
Observer



状态
State



策略
Strategy



模版方法
Template Method



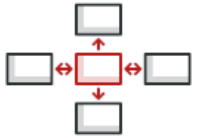
访问者
Visitor



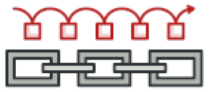
解释器
Interpreter



备忘录
Memento



中介
Mediator



迭代器
Iterator

行为解耦



责任链
Chain of Responsibility



命令
Command

引言：六种创建型模式的具体作用-对象生成与使用解耦

序号	设计模式	一句话归纳	目的
1	单例模式 Singleton Pattern	一个类只允许创建一个对象	在牺牲部分灵活性的前提下，通过严格的实例控制，解决资源共享与一致性难题
2	简单工厂 Simple Factory Pattern	封装对象的创建逻辑，降低客户端代码与具体产品类之间的耦合度	屏蔽内部的对象创建细节，提高扩展灵活性，统一创建接口
3	工厂方法模式 Factory Method Pattern	将对象的创建过程延迟到子类，让子类决定实例化哪个具体类	解耦对象的创建逻辑与使用逻辑，提升代码的灵活性和扩展性
4	抽象工厂 Abstract Factory Pattern	通过统一的接口创建一组相互关联或依赖的产品对象，隐藏具体类的实例化逻辑	一个工厂负责创建多个不同类型的对象，有效地减少工厂类的个数
5	建造者模式 Builder Pattern	将复杂对象的构建过程与其表示形式分离，使相同的构建逻辑能生成不同结构的对象	解耦构建逻辑与对象结构，简化复杂对象构造，提升扩展性
~~6~~	~~原型模式~~ ~~Prototype Pattern~~	~~复制现有对象来创建新对象~~	~~避免复杂的初始化过程，提高对象创建效率~~

1. 单例模式

Singleton Pattern

1.1 引例：资源访问冲突处理-日志打印覆盖问题

```
// 往文件中打印日志，请判断代码是否存在问题。
public class Logger {  //低层模块
    private FileWriter writer;
    public Logger() {
        File file = new File("/Users/xgd/log.txt");
        writer = new FileWriter(file, true); //true表示追加写入
     }
    public void log(String message) {
        writer.write(message);
    }
}
```

```
// Logger类的应用1：
public class UserController { //高层模块
    private Logger logger = new Logger();
    public void login(String username, String password) {
        // ...省略业务逻辑代码...
        logger.log(username + " logined!");
    }
}
```

1.1 引例：资源访问冲突处理-日志打印覆盖问题

```
// Logger类的应用2：
public class OrderController { //高层模块
    private Logger logger = new Logger();
    public void create(OrderVo order) {
        // ...省略业务逻辑代码...
        logger.log("Created an order: " + order.toString());
    }
}
```

- Logger，UserController，OrderController 这三段代码有什么问题?

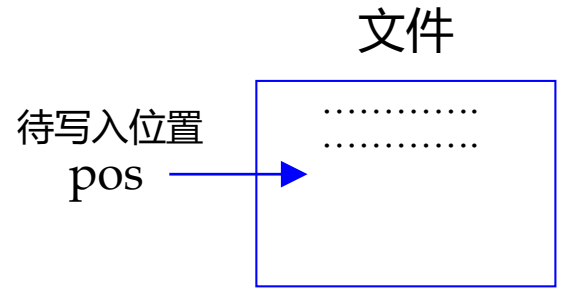
我们注意到：所有的日志都写入到同一个文件。在 UserController 和 OrderController 中，我们分别创建两个 Logger 对象。

在 多线程环境下，如果两个线程同时分别执行 login() 和 create() 两个函数，并且同时将日志写到同一个 log.txt 文件中，那就有可能存在日志信息的覆盖。

1.1 引例：资源访问冲突处理–日志打印覆盖问题

```
int j = 5;
void increase() {
    j = j + 1;
}
```

类比理解：




```
class File {
    int fid;
    int pos;
}
```

CPU 调度执行顺序

线程 1 执行 increase	线程 2 执行 increase
读取 j 值 (5)	
	读取 j 值 (5)
执行 j = j + 1；　(j = 6)	
	执行 j = j + 1；　(j = 6)

最终 j 值等于 6，而非 7

CPU 调度执行顺序

线程 1 写入文件	线程 2 写入文件
读取 pos = 105；	
	读取 pos = 105；
在pos = 105 处写入“abc”	
	在 pos = 105 处写“def”

最终线程 2 写入的“def”覆盖了线程 1 写入的“abc”

1.2 尝试解决：给 log() 函数加互斥锁，是否能解决问题?

```
//往文件中打印日志
public class Logger {
    private FileWriter writer;
    public Logger() {
        File file = new File("/Users/xgd/log.txt");
        writer = new FileWriter(file, true); //true表示追加写入
    }
    public void log(String message) {
        synchronized (this) { //加上 对象级别 的互斥锁，是否能解决问题?
            writer.write(message);
        }
    }
}
```

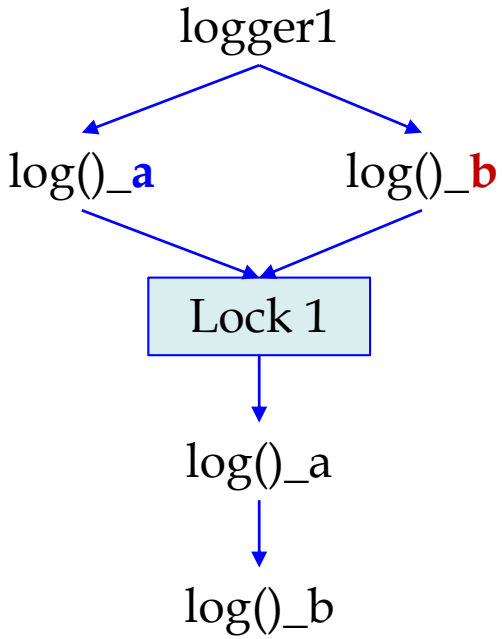
对象级别的锁

答案是否定的，原因：UserController，OrderController 分别生成一个 Logger 对象，不同的对象之间并不共享同一把锁。

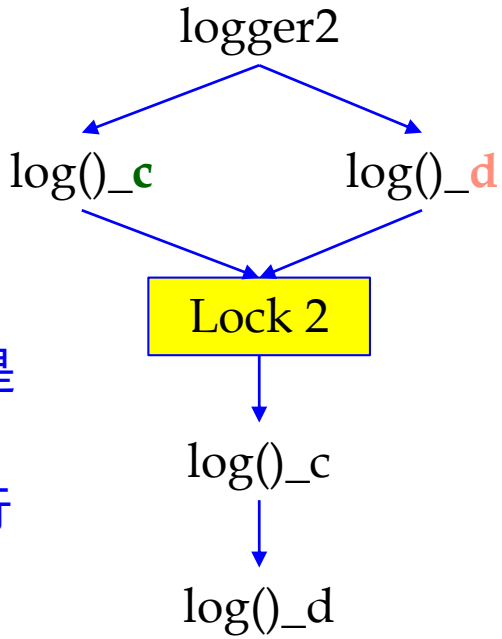
不同的线程下，通过不同的对象调用执行 log() 函数，对象级别的锁并不会起作用，仍然有可能存在写入日志互相覆盖的问题

1.2 尝试解决：给 log() 函数加互斥锁，是否能解决问题?

对象 logger1 多线程并发执行 2 次 log() 示意



对象 logger2 多线程并发执行 2 次 log() 示意



未能解决，原因：Lock1 和 lock2 是不同的锁， log()_a、 log()_b 与 log()_c、 log()_d 无法保证顺序执行

Lock1 强制 log()_a、 log()_b 顺序执行

Lock2 强制 log()_c、 log()_d 顺序执行

1.2 尝试解决：给 log() 函数加互斥锁，是否能解决问题?

```java
//个往文件中打印日志
public class Logger {
    private FileWriter writer;
    public Logger() {
        File file = new File("/Users/xgd/log.txt");
        writer = new FileWriter(file, true); //true表示追加写入
    }
    public void log(String message) {
        synchronized (Logger.class) { // 类级别的锁
            writer.write(message);
        }
    }
}
```

类级别的锁

只需要把对象级别的锁，换成类级别的锁就可以了。让所有的对象都共享同一把锁。这样就避免了不同对象之间同时调用 log() 函数，而导致的日志覆盖问题。

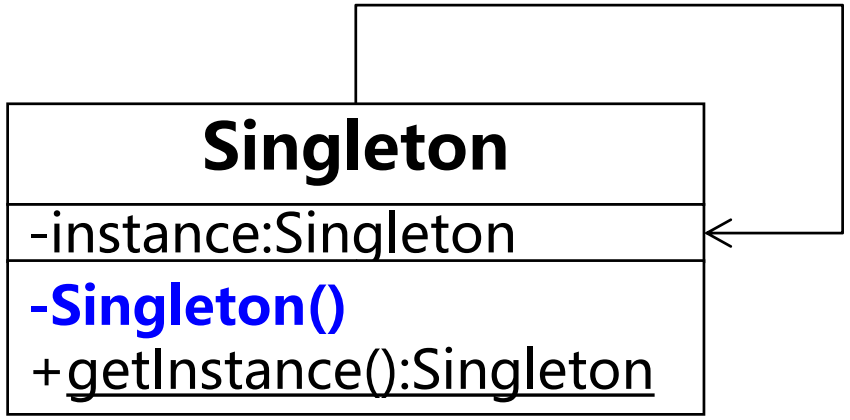
另一种思路：如果系统中只有且一个 Logger 对象，也能解决问题。

```java
//往文件中打印日志
public class Logger {
    private FileWriter writer;
    private static final Logger instance = new Logger();  //私有的常量，设置之后不能动
    public Logger() {
        File file = new File("/Users/xgd/log.txt");
        writer = new FileWriter(file, true); //true表示追加写入
     }
    public static Logger getInstance() {   //通过该静态方法访问唯一的 instance常量
        return instance;
    }
    public void log(String message) {
            writer.write(message);
    }
}
```

```java
// Logger类的应用:
public class OrderController {
    public void create(OrderVo order) {
        // ...省略业务逻辑代码...
        Logger.getInstance().log("Created an order: " + order.toString()); //由静态方法取对象
    }
}
```

1.3 单例模式（Singleton Pattern）的定义及典型结构

- 一个类只允许创建一个对象，那这个类就是一个单例类，这种设计模式就叫作单例设计模式，简称单例模式。



单例模式的典型结构

- 要实现一个单例，需要关注的点无外乎下面几个
  - ✓ 构造函数需要是 private 访问权限的，这样才能避免外部通过 new 创建实例；
  - ✓ 考虑对象创建时的线程安全问题；
  - ✓ 考虑是否支持延迟加载；
  - ✓ 考虑 getInstance() 性能是否高（是否加锁）。

1.4 单例模式常见的几种实现方式-饿汉式

- 饿汉式的实现方式是在类加载的时候，instance 静态实例就已经创建并初始化，而 JVM 类加载机制保证同一类只会被加载一次，所以 instance 实例的创建过程是线程安全的。

```
// id生成器
public class IdGenerator {
    // AtomicLong是一个Java并发库中提供的一个原子变量类型
    // 它将一些线程不安全需要加锁的复合操作封装为了线程安全的原子操作
     private AtomicLong id = new AtomicLong(10000); //java.util.concurrent.atomic 包下

    private static final IdGenerator instance = new IdGenerator();  //饿了，直接初始化

    public static IdGenerator getInstance() {
        return instance;
    }

    public long getId() {
        return id.incrementAndGet();
    }
}
```

1.4 单例模式常见的几种实现方式–懒汉式

- 懒汉式相对于饿汉式的优势是支持延迟加载，即在真正用到 IdGenerator 的时候，再创建实例

```
// id生成器
public class IdGenerator {
    private AtomicLong id = new AtomicLong(0);
    private static IdGenerator instance;

    public static synchronized IdGenerator getInstance() {
        if (instance == null) {
            instance = new IdGenerator(); // 懒汉，拖延到用对象的时候，再创建
        }
        return instance;
    }

    public long getId() {
        return id.incrementAndGet();
    }
}
```

getInstance() 这个方法加了一把大锁（Synchronized）。导致这个函数的并发度很低(=1, 相当于串行)。
因此，如果频繁地用到 getInstance，那么频繁加锁、释放锁及并发度低等问题，会导致性能瓶颈

1.4 单例模式常见的几种实现方式–双重检测 double check

```
public class IdGenerator {
    private AtomicLong id = new AtomicLong(0);
    private static IdGenerator instance;

    public static IdGenerator getInstance() {
        if (instance == null) {  //check 解决性能问题
            synchronized (IdGenerator.class) { // 此处为类级别的锁
                  if (instance == null) { //check 保证生成对象的唯一性
                     instance = new IdGenerator();
                  }
            }
        }
        return instance;
    }

    public long getId() {
        return id.incrementAndGet();
    }
}
```

饿汉式不支持延迟加载，懒汉式有性能问题，不支持高并发。

既支持延迟加载、又支持高并发的单例实现方式：双重检测实现方式

instance 被创建之后，再调用 getInstance() 函数，该函数不会再进入到加锁逻辑中。

所以，这种实现方式解决了懒汉式并发度低的问题。

1.4 单例模式常见的几种实现方式-静态内部类实现

```
public class IdGenerator {
    private AtomicLong id = new AtomicLong(0);

    private static class SingletonHolder {
        private static final IdGenerator instance = new IdGenerator();
    }

    public static IdGenerator getInstance() {
        return SingletonHolder.instance;
    }

    public long getId() {
        return id.incrementAndGet();
    }
}
```

当外部类 IdGenerator 被加载的时候，并不会创建内部类 SingletonHolder 实例对象。只有当调用 getInstance() 方法时，SingletonHolder 才会被加载，这个时候才会创建 instance

instance 的唯一性、创建过程的线程安全性，都由 JVM 来保证。所以，这种实现方法既保证了线程安全，又能做到延迟加载

1.4 单例模式常见的几种实现方式-枚举实现

```
public enum IdGenerator {

    INSTANCE;

    private AtomicLong id = new AtomicLong(0);
    public long getId() {
        return id.incrementAndGet();
    }
}
```

```
@Test
public void testGetInstance() {
    IdGenerator idGenerator = IdGenerator.INSTANCE;
    System.out.println(idGenerator.getId());
}
```

这种实现方式利用了 Java 枚举类型本身的特性。保证了实例创建的线程安全性（不可变类型，不能被继承，内部成员变量都是 private，只读等）和实例的唯一性（每个枚举项在 JVM 中仅有一个实例）。

1.5 单例模式小结

单例设计模式（Singleton Design Pattern）的定义：一个类只允许创建一个对象，那这个类就是一个单例类，这种设计模式就叫作单例设计模式，简称单例模式。

用处：从业务概念上，有些数据在系统中只应该保存一份，就比较适合设计为单例类。比如，系统的配置信息类。除此之外，我们还可以使用单例解决资源访问冲突的问题。在 Spring 框架中，bean 默认都是单例

实现方式	一句话归纳	优点	缺点
饿汉式	类加载的期间实例化	线程安全	不支持延迟加载
懒汉式	用到实例的时候再产生对象。	支持延迟加载	并发度低
双重检测	实例化时加锁，后续不进入到加锁逻辑中	支持延迟加载、高并发	
静态内部类	利用 Java 的静态内部类来实现单例	支持延迟加载，支持高并发，实现简单	
枚举	最简单的实现方式，基于枚举类型的单例实现。充分利用Java枚举的特性	线程安全性，实现最简单	

2. 简单工厂模式
Simple Factory Pattern

2.1 引例：配置文件解析

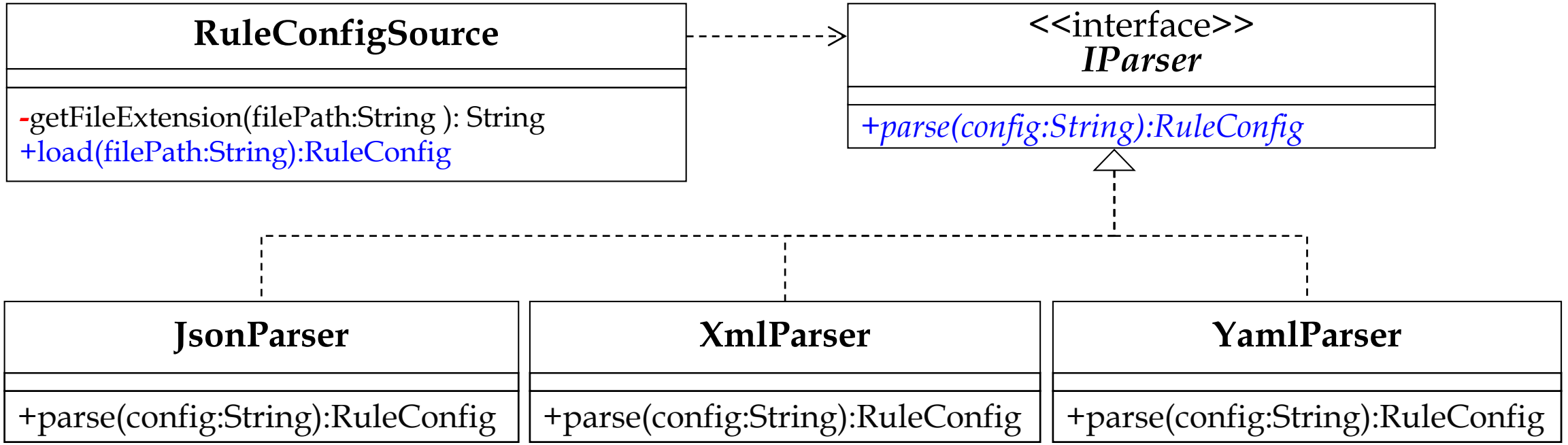
- 某大型持续项目需要对配置文件进行解析，以便获取系统相关的配置信息。我们需要根据配置文件的后缀（json、xml、yaml），选择不同的解析器，将存储在文件中的配置解析成内存对象 RuleConfig。比如：
  - Json 类型的文件需要用 Json 解析器 JsonParser，解析成 RuleConfig 对象；
  - Xml 类型文件使用 Xml 解析器 XmlParser，解析成 RuleConfig 对象；
  - 以此类推…

2.1 引例：配置文件解析

```
public class RuleConfigSource {
    public RuleConfig load(String filePath) {
        String fileExtension = getFileExtension(filePath);
        IParser parser = null;
        if (“json”.equalsIgnoreCase(fileExtension)) {
            parser = new JsonParser();
        } else if (“xml”.equalsIgnoreCase(fileExtension)) {
            parser = new XmlParser();
        } else {
            throw new InvalidRuleConfigException(//…)
        }
        String configText = “”;
        //从filePath文件中读取配置文本到configText中，代码略…
        //继续完成下面的代码
        RuleConfig ruleConfig = parser.parse(configText);
        return ruleConfig;
    }
    private String getFileExtension(String filePath) {
        //...解析文件名获取扩展名，比如 rule.json，返回json
        return "json";
    }
}
```

2.1 引例：配置文件解析

- 为了更清晰了解代码结构，进一步了解设计意图，画类图分析如下。



IParser 及其实现类已经形成层次关系，可以实现多态；而 RuleConfigSource 中的 load() 方法，明显职责过多，应该按照单一职责进行重构

2.1 引例：配置文件解析

```
public class RuleConfigSource {
    public RuleConfig load(String filePath) {
        String fileExtension = getFileExtension(filePath);
        IParser parser = null;
        if (“json”.equalsIgnoreCase(fileExtension)) {
            parser = new JsonParser();
        } else if (“xml”.equalsIgnoreCase(fileExtension)) {
            parser = new XmlParser();
        } else {
            throw new InvalidRuleConfigException(//…)
        }
        String configText = “”;
        //从 filePath文件中读取配置文本到 configText中，代码略…
        //继续完成下面的代码
         RuleConfig ruleConfig = parser.parse(configText);
        return ruleConfig;
    }
    private String getFileExtension(String filePath) {
        //...解析文件名获取扩展名，比如 rule.json，返回json
        return "json";
    }
}
```

这部分代码的目的就是为了生成一个对应文件类型的解析器

根据单一职责，这部分代码应从load方法中剥离，并封装成一个函数

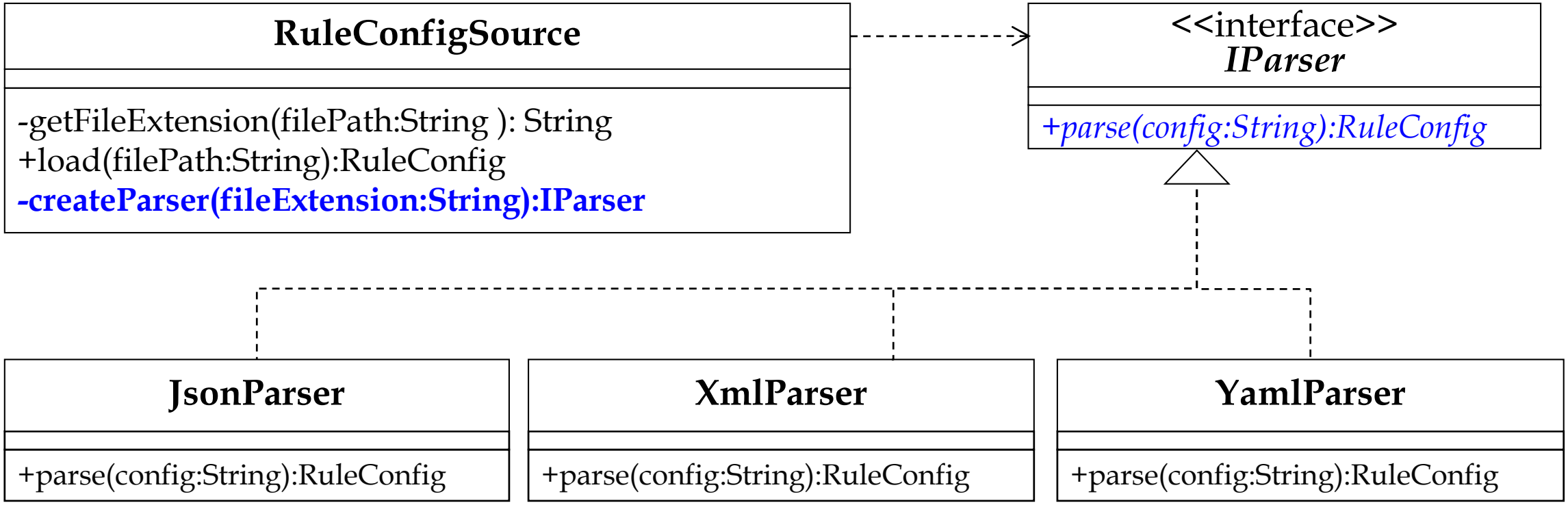
```
public RuleConfig load(String filePath) {
    String fileExtension = getFileExtension(filePath);
    IParser parser = createParser(fileExtension);
    if (parser == null) {
        throw new InvalidRuleConfigException();
    }
    String configText = "";
    //从 filePath文件中读取配置文本到 configText中，代码略...
    RuleConfig ruleConfig = parser.parse(configText);
    return ruleConfig;
}

private String getFileExtension(String filePath) {
    //...解析文件名获取扩展名，比如 rule.json，返回 json
    return "json";
}
```

```
private IParser createParser(String fileExtension) {  //抽取为只产生对象的方法
    IParser parser = null;
    if ("json".equalsIgnoreCase(fileExtension)) {
        parser = new JsonParser();
    } else if ("xml".equalsIgnoreCase(fileExtension)) {
        parser = new XmlParser();
    }
    return parser;
}
```

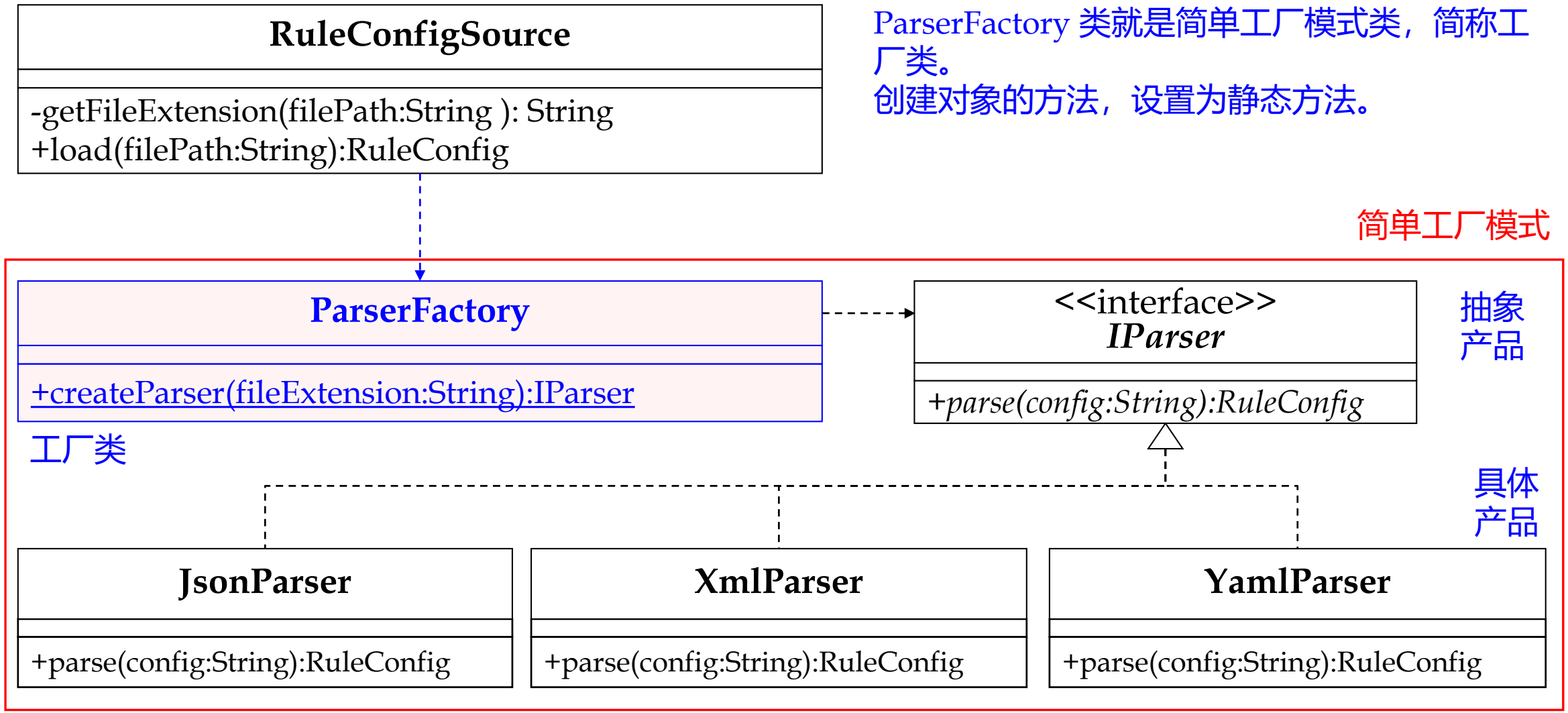
配置文件解析器的代码被分离到此

2.1 引例：配置文件解析



可以让类的职责更加单一、代码更加清晰，我们进一步将 createParser() 函数剥离，并封装到一个独立的类中，让这个类只负责对象的创建

2.2 简单工厂模式 Simple Factory Pattern



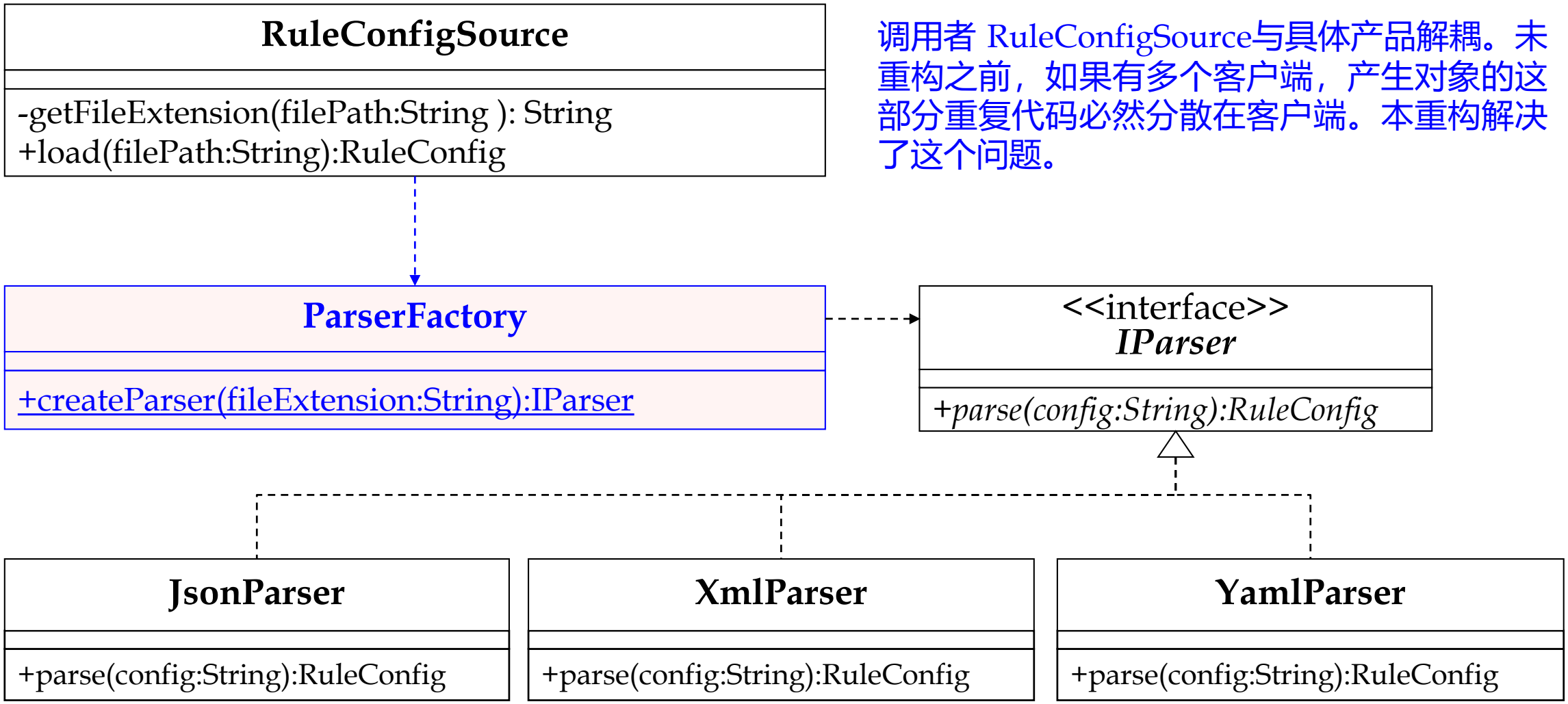
2.3 简单工厂模式的代码-实现方式一

```
public class ParserFactory {
        public static IParser createParser(String fileExtension) {
        IParser parser = null;

        if ("json".equalsIgnoreCase(fileExtension)) {
            parser = new JsonParser();
        } else if ("xml".equalsIgnoreCase(fileExtension)) {
            parser = new XmlParser();
        } else if ("yaml".equalsIgnoreCase(fileExtension)) {
            parser = new YamlParser();
        } else if ("properties".equalsIgnoreCase(fileExtension)) {
            parser = new PropertiesParser();
        }

        return parser;
    }
}
```

2.3 简单工厂模式的代码-方式一只是 if…else 转移?



调用者 RuleConfigSource与具体产品解耦。未重构之前，如果有多个客户端，产生对象的这部分重复代码必然分散在客户端。本重构解决了这个问题。

2.3 简单工厂模式的代码-实现方式一存在的问题

```
public class RuleConfigSource { //上层模块（调用者）
    public RuleConfig load(String filePath) {
        String fileExtension = getFileExtension(filePath);

        IParser parser = ParserFactory.createParser(fileExtension);

        if (parser == null) {
            throw new InvalidRuleConfigException();
        }
        String configText = "";
        //从ruleConfigFilePath文件中读取配置文本到configText中
         RuleConfig ruleConfig = parser.parse(configText);
        return ruleConfig;
    }

    private String getFileExtension(String filePath) {
        //...解析文件名获取扩展名，比如rule.json，返回json
        return "json";
    }
}
```

每次创建一个新的 parser，浪费算力，如果将对象缓存呢?

2.3 简单工厂模式的代码-实现方式二

```
public class ParserFactory {

    private static final Map<String, IParser> cachedParsers = new HashMap<>(); // 缓存

    static { // 类加载的时候创建
        cachedParsers.put("json", new JsonfigParser());
        cachedParsers.put("xml", new XmlParser());
        cachedParsers.put("yaml", new YamlParser());
        cachedParsers.put("properties", new PropertiesParser());
    }

    public static IParser createParser(String fileExtension) {
        if (fileExtension == null || fileExtension.isEmpty()) {
            return null; //返回null还是IllegalArgumentException全凭你自己说了算
         }
        IParser parser = cachedParsers.get(fileExtension.toLowerCase());
        return parser;
    }
}
```

查表法

2.4 简单工厂模式讨论-简单工厂是否满足开闭原则

```
public class ParserFactory { //实现方式一

    public static IParser createParser(String fileExtension) {

        IParser parser = null;

        if ("json".equalsIgnoreCase(fileExtension)) {
            parser = new JsonParser();
        } else if ("xml".equalsIgnoreCase(fileExtension)) {
            parser = new XmlParser();
        } else if ("yaml".equalsIgnoreCase(fileExtension)) {
            parser = new YamlParser();
        } else if ("properties".equalsIgnoreCase(fileExtension)) {
            parser = new PropertiesParser();
        }

        return parser;
    }
}
```

If..else 都是针对具体编程，配置文件类型变化，必导致该段代码修改，不满足 OCP

2.4 简单工厂模式讨论-简单工厂是否满足开闭原则

```
public class ParserFactory { //实现方式二

    private static final Map<String, IParser> cachedParsers = new HashMap<>(); // 缓存

    static { // 类加载的时候创建
        cachedParsers.put("json", new JsonParser());
        cachedParsers.put("xml", new XmlParser());
        cachedParsers.put("yaml", new YamlParser());
        cachedParsers.put("properties", new PropertiesParser());
    }

    public static IParser createParser(String fileExtension) {
        if (fileExtension == null || fileExtension.isEmpty()) {
            return null; //返回null还是IllegalArgumentException全凭你自己说了算
         }
        IParser parser = cachedParsers.get(fileExtension.toLowerCase());
        return parser;
    }
}
```

新增/删除配置文件类型，必然修改该类代码。不满足 OCP

3. 工厂方法模式
Factory Method Pattern

3.1 简单工厂不满足开闭原则的根本原因分析

```
public class ParserFactory {

    public static IParser createParser(String fileExtension) {
        IParser parser = null;
        if ("json".equalsIgnoreCase(fileExtension)) {
            parser = new JsonParser();
        } else if ("xml".equalsIgnoreCase(fileExtension)) {
            parser = new XmlParser();
        } else if ("yaml".equalsIgnoreCase(fileExtension)) {
            parser = new YamlParser();
        } else if ("properties".equalsIgnoreCase(fileExtension)) {
            parser = new PropertiesParser();
        }
        return parser;
    }
}
```

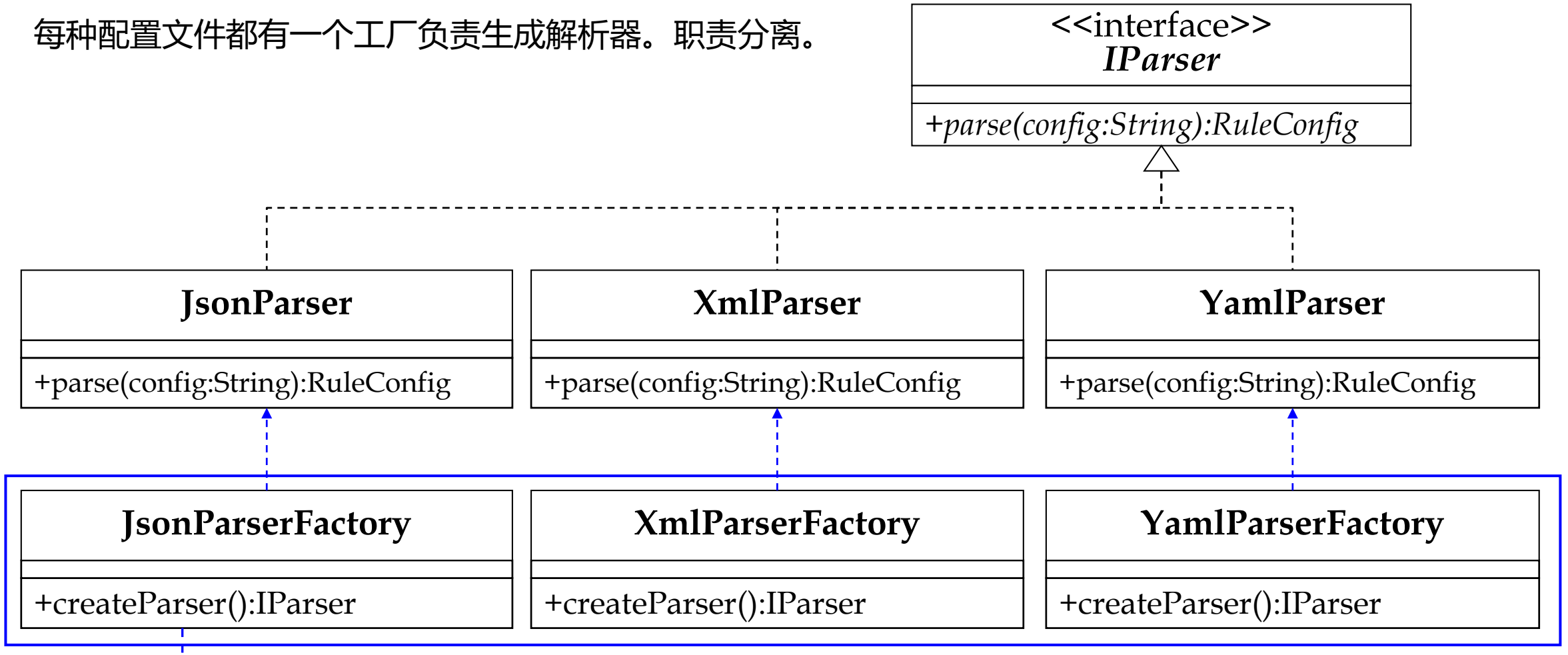
职责过多，产生对象时需要考虑每一种具体情况，必然需要 if..else

如何分离职责？

每种配置文件一个工厂！

3.2 为每种配置文件设置一个专门的工厂，实现职责分离

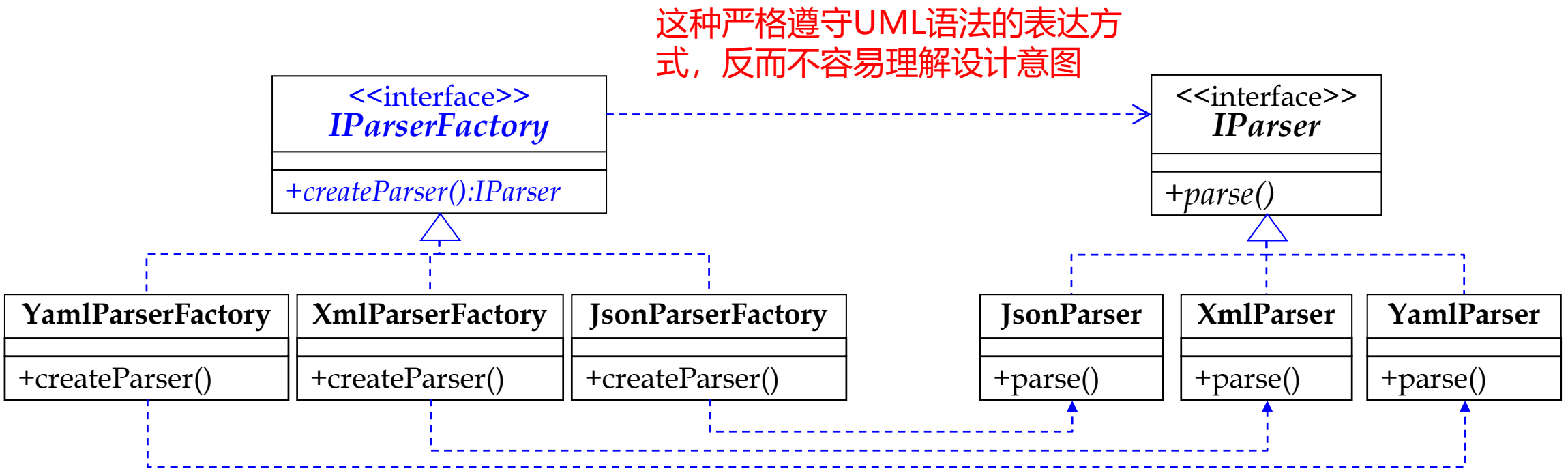
每种配置文件都有一个工厂负责生成解析器。职责分离。



createParser() 方法无需输入参数，不必判断类型：直接 return new JsonParser(); 其他的具体工厂亦如是 JsonParserFactory 等均为具体类，碰到具体要抽象，应进一步抽象出工厂接口类

3.2 工厂方法模式-严格遵守 UML 语法的表达方式

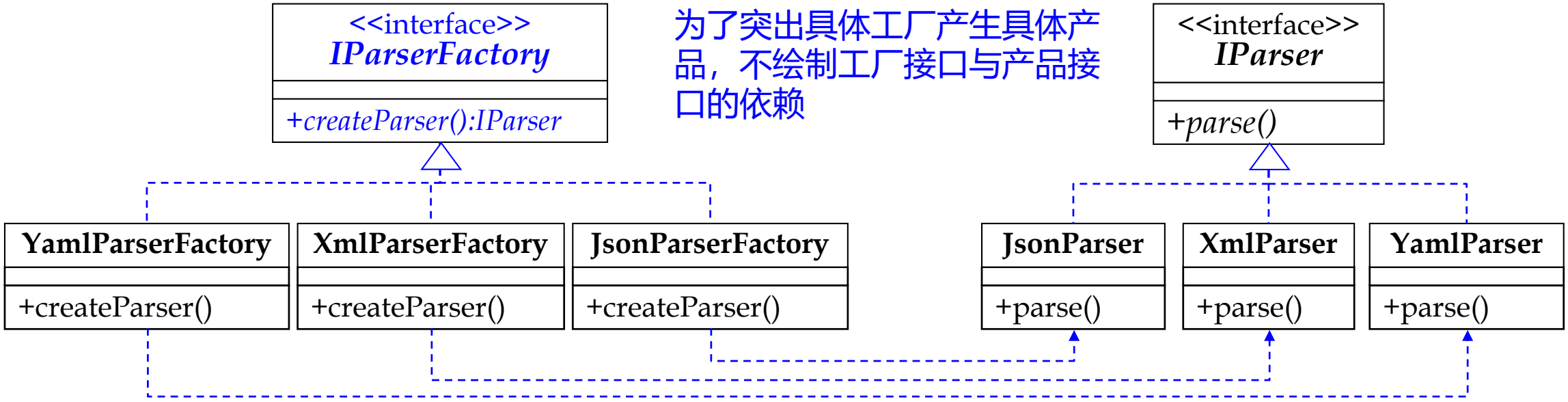
- 为了节省页面空间，将各个类中的方法去掉返回值及参数（实际参数及返回值参考前页）



符合开闭原则：无论是产品类，还是工厂类的变更都不会影响已存在的代码

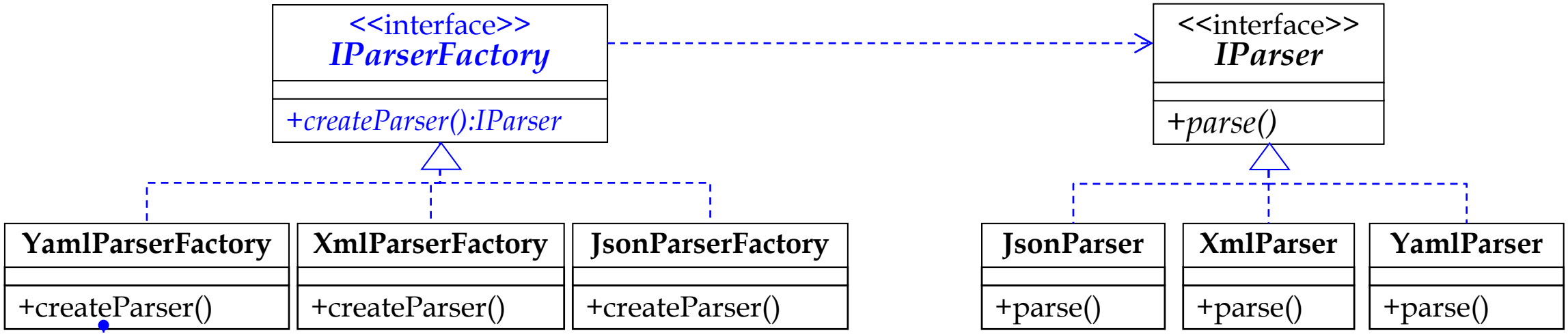
3.2 工厂方法模式-类图表达方式一

- 为了节省页面空间，将各个类中的方法去掉返回值及参数（实际参数及返回值参考前页）



符合开闭原则：无论是产品类，还是工厂类的变更都不会影响已存在的代码

3.2 工厂方法模式-类图表达方式二



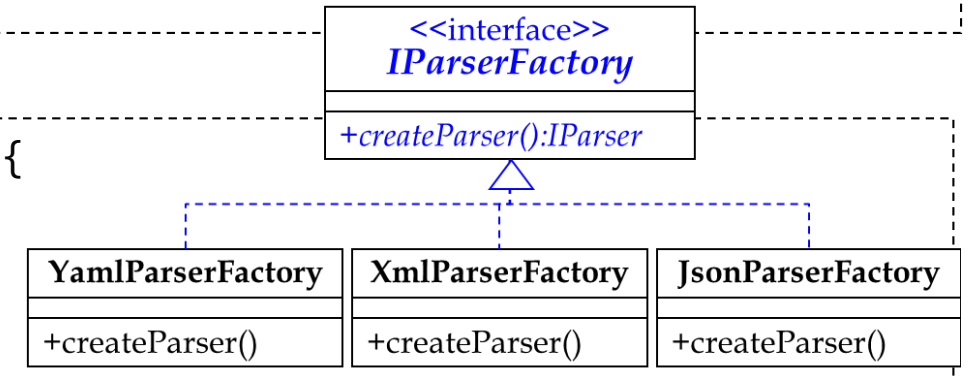

```
public IParser createParser() {
    return new XmlParser();
}
```

此种表达方式，也能清晰表达具体工厂产生具体产品。此时不必绘制具体类和具体产品的依赖关系。类图表达方式二更简洁，建议采用这种方式。

3.2 工厂方法模式-代码实现

```java
public interface IParserFactory {
    IParser createParser();
}
```

```java
public class JsonParserFactory implements IParserFactory {
    @Override
    public IParser createParser() {
        return new JsonParser();
    }
}
```




```java
public class XmlParserFactory implements IParserFactory {
    @Override
    public IParser createParser() {
        return new XmlParser();
    }
}
```

课堂练习1：女娲造人

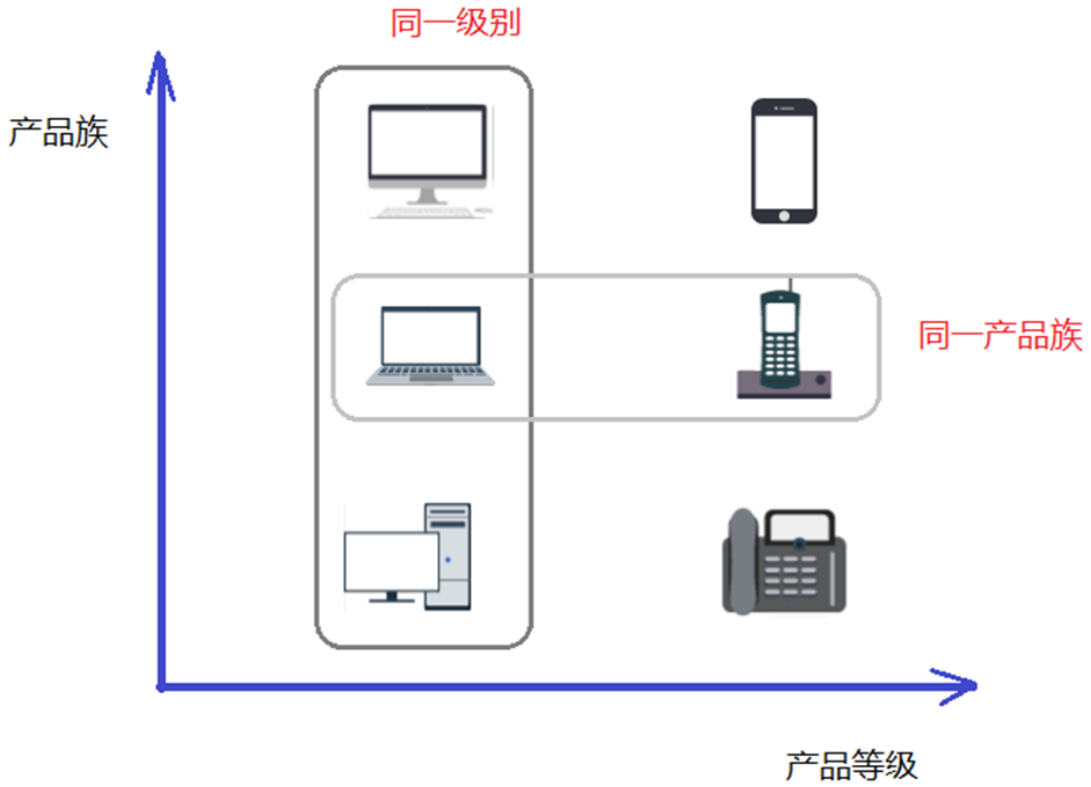
- 俗说天地开辟，未有人民，女娲抟黄土做人。剧务，力不暇供，乃引绳于泥中，举以为人。绳分阴阳，人分男女，阳绳造男，阴绳造女。
- 要求：
  - 绘制女娲用阴、阳绳造人的这一场景的类图，要求满足开闭原则
- 名词命名：
  - 女娲：NvWa　人：Human　男人：Man　女人：Woman　绳子：Rope
  - 阴绳：Yin Rope　阳绳：Yang Rope
  - 男人女人都有的行为（男女有别）：吃饭：eat　干活：work

4. 抽象工厂模式
Abstract Factory Pattern

4.1 需求变更

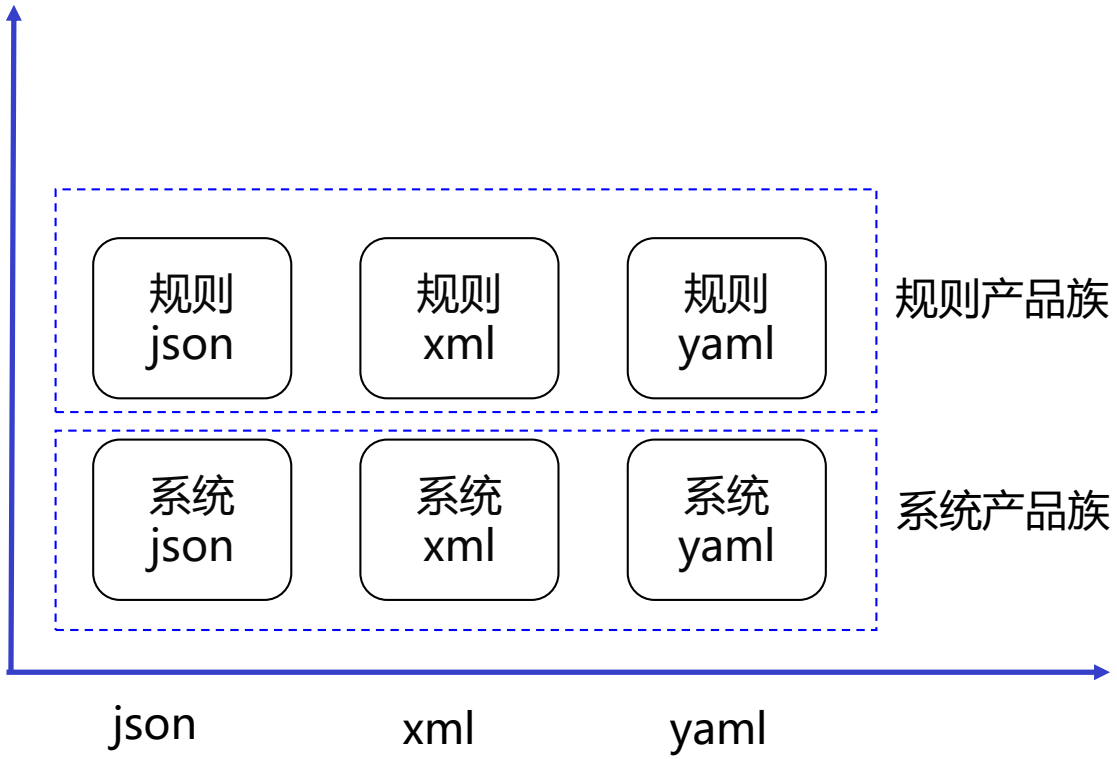
- 某大型持续项目需要对配置文件进行解析，以便获取相关的配置信息。现在有两种配置方式：
  - 基于规则的配置 RuleConfig（已经在工厂模式中实现过）
  - 基于系统的配置 SystemConfig。
- 未来可能有基于业务的配置 IBizConfig。
- 每种配置方式都有 json、xml、yaml等类型的的配置文件。需要我们针对不同配置文件进行解析，将存储在配置文件中的配置信息解析成规则配置对象 RuleConfig 或者 系统配置对象 SystemConfig
- 例如：针对 SystemConfig
  - 基于系统的配置： json格式的配置文件，需要 JsonSysConfigParser
  - 基于规则的配置： json格式的配置文件，需要 JsonRuleConfigParser

4.2 产品等级和产品族的划分

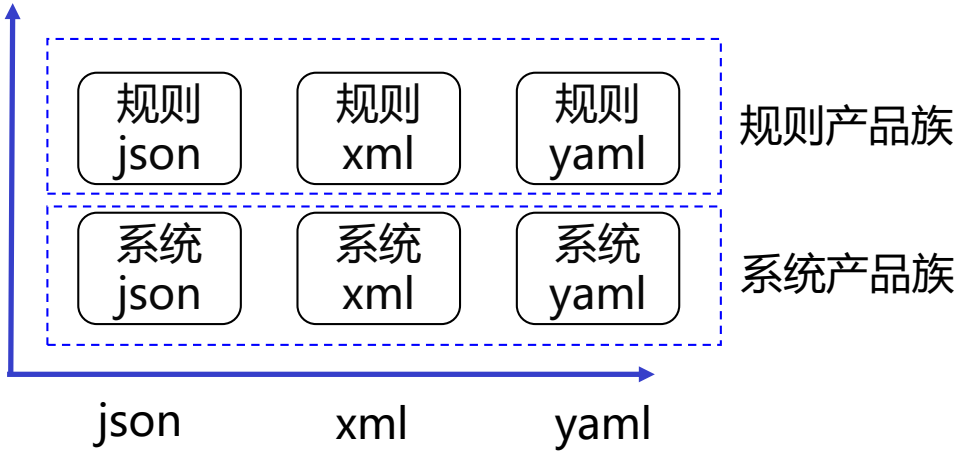


生活中的产品等级和产品族划分

基于左图产品族和产品等级，需求可以划分为：

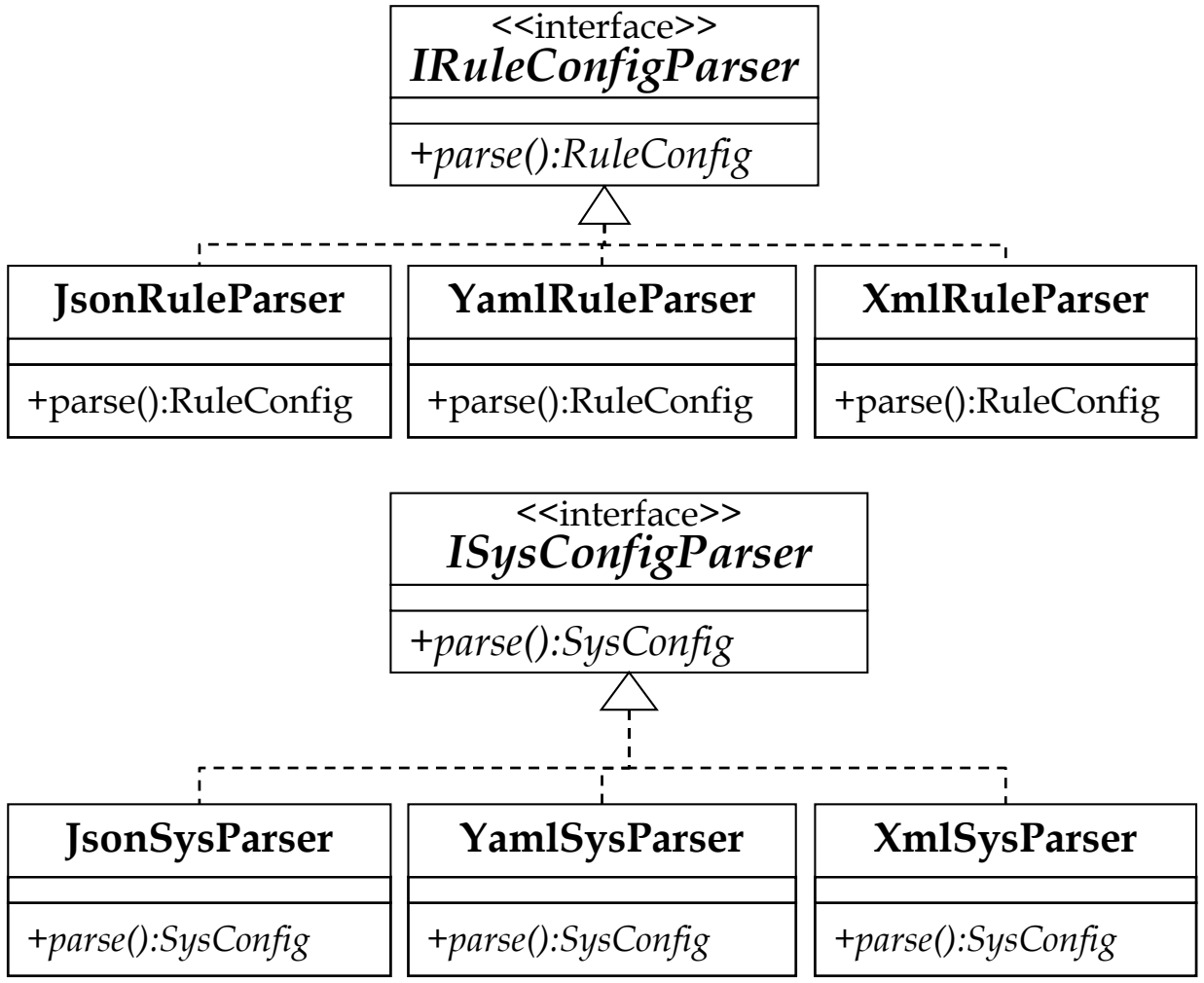


4.3 抽象工厂解决的问题



两个产品族，各具有三种 parser ，也就是6种 parser。每个具体的 parser 都编写一个工厂类，要编写 6 个工厂类。
工厂类太多了。过多的类让系统难维护

让一个工厂负责创建多个不同类型的对象，这样就可以有效地减少工厂类的个数



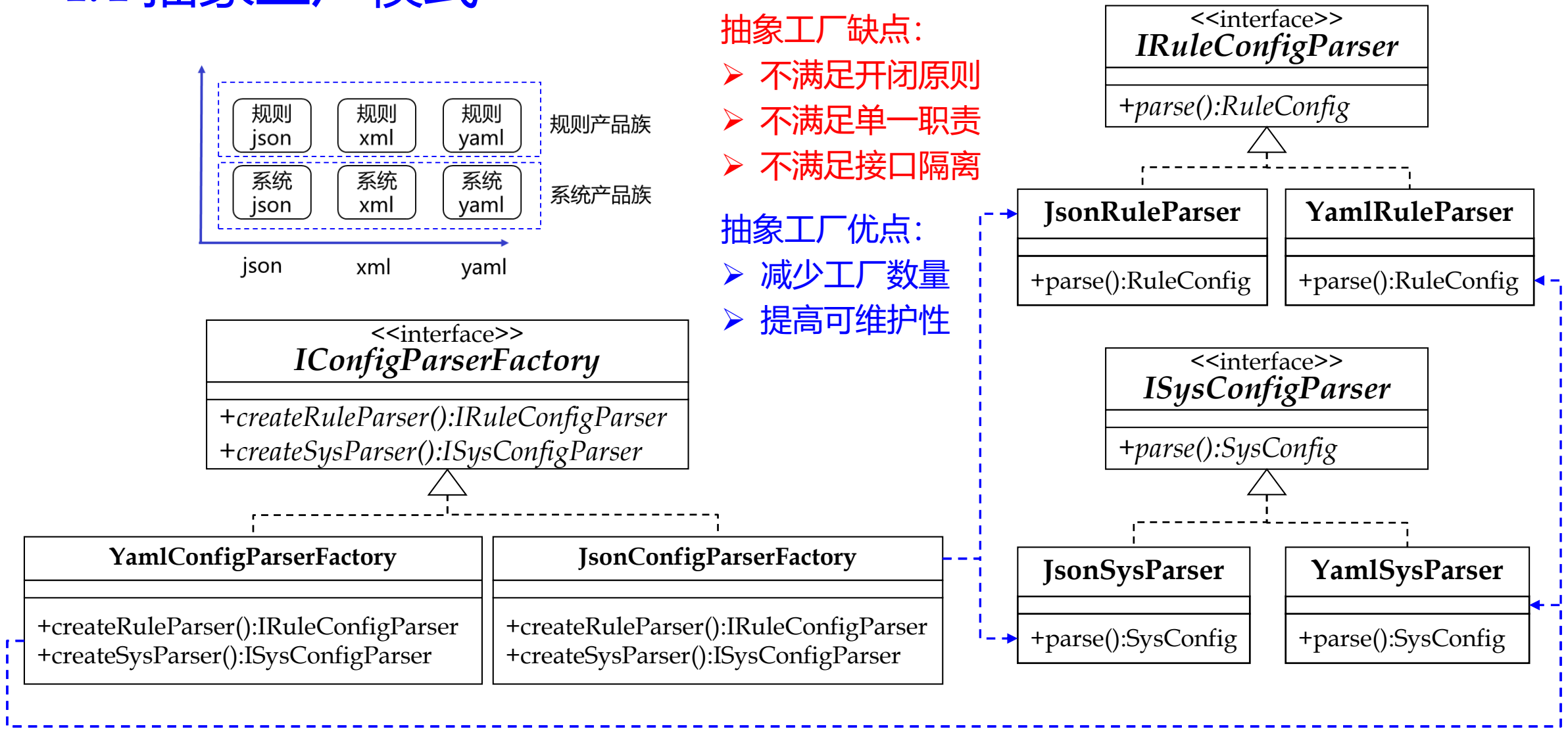
4.4 抽象工厂模式

抽象工厂缺点：

- 不满足开闭原则
- 不满足单一职责
- 不满足接口隔离

抽象工厂优点：

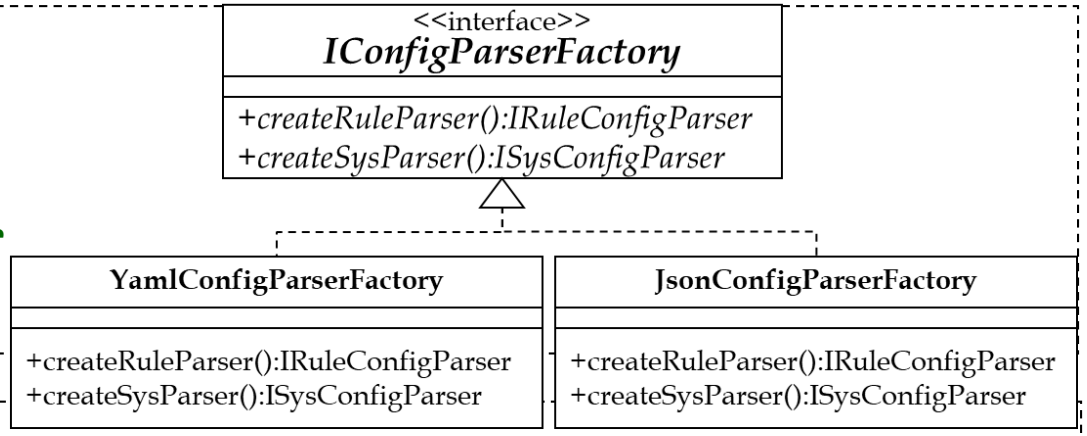
- 减少工厂数量
- 提高可维护性



抽象工厂模式可以看成是一种反模式，是在提高可维护性和维持设计原则的一种折中和平衡（个人观点）

4.5 抽象工厂模式的代码实现

```
public interface IConfigParserFactory {
    IRuleConfigParser createRuleParser();
    ISystemConfigParser createSystemParser();
    //此处可以扩展新的parser类型，比如IBizConfigParser
}
```

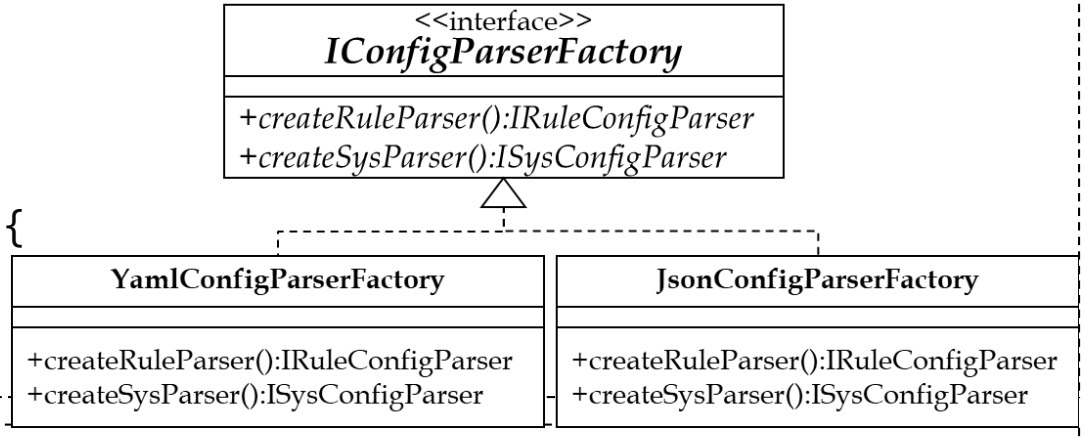



```
public class JsonConfigParserFactory implements IConfigParserFactory {
    @Override
    public IRuleConfigParser createRuleParser() {
        return new JsonRuleConfigParser();
    }

    @Override
    public ISystemConfigParser createSystemParser() {
        return new JsonSystemConfigParser();
    }
}
```

4.5 抽象工厂模式的代码实现

```
public class YamlConfigParserFactory implements IConfigParserFactory {
    @Override
    public IRuleConfigParser createRuleParser() {
        return new YamlRuleParser();
    }
    @Override
    public ISystemConfigParser createSystemParser() {
        return new YamlSysParser();
    }
}
```




```
public class Client {
    public static void main(String[] args) {
        IConfigParserFactory configParserFactory = new JsonConfigParserFactory();
        IRuleConfigParser ruleParser = configParserFactory.createRuleParser();
        ISystemConfigParser systemParser = configParserFactory.createSystemParser();
        System.out.println(ruleParser);
        System.out.println(systemParser);
    }
}
```

```
edu.hitwh.absfactory.JsonRuleConfigParser@448139f0
edu.hitwh.absfactory.JsonSystemConfigParser@7cca494b
```

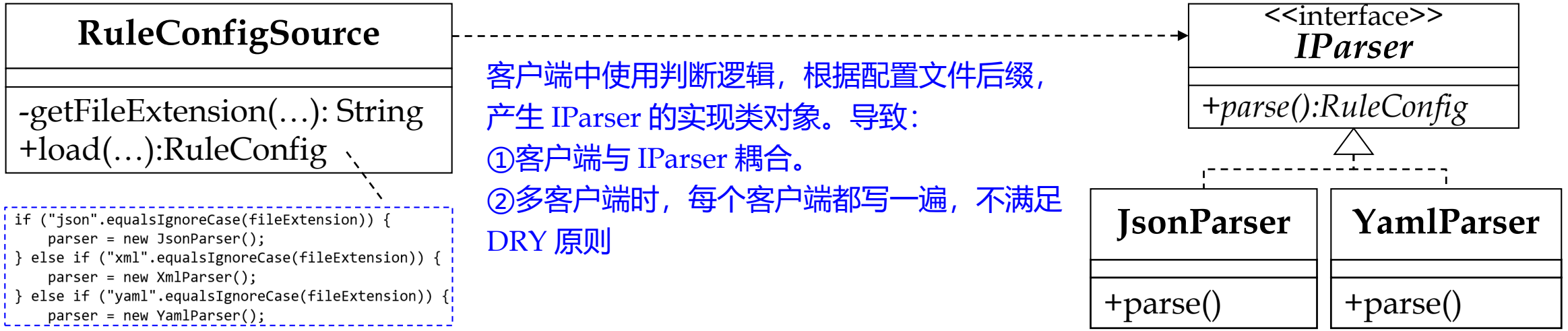
练习2：女娲造万物

- 女娲将举绳造人的方法推广到创建各种动物上。将绳子搅到泥水里，然后将沾满泥水的绳子凭空一甩，象人的变成了人，象动物的变成了动物。阴绳造出的是女人和雌性动物，阳绳造出男人和雄性动物。
- 要求：
  - 绘制女娲用阴阳绳造万物的类图；
- 名词命名参考：
  - 女娲：NuWa　人：Human　男人：Man　女人：Woman　绳子：Rope；动物：animal，雄性动物 male animal；雌性动物 female animal
  - 阴绳：Yin Rope　阳绳：Yang Rope
  - 男人女人/雄性动物和雌性动物 都有的行为（男女、雄雌有别）：吃：eat　干活：work

5. 从简单工厂到抽象工厂-设计复盘及讨论

From Simple Factory to Abstract Factory - Design Review and Discussion

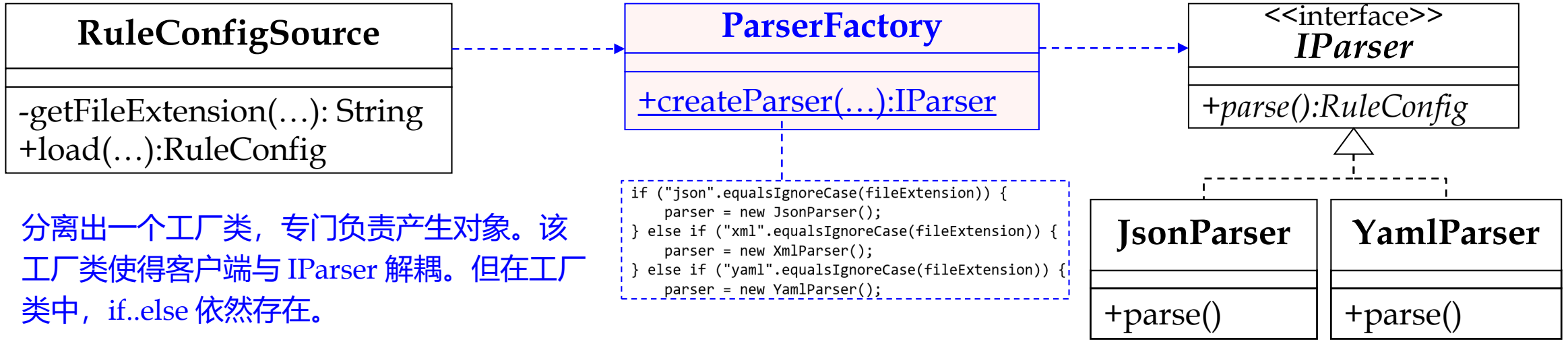
5.1 简单工厂-解耦客户端与具体产品类



客户端中使用判断逻辑，根据配置文件后缀，产生 IParser 的实现类对象。导致：

①客户端与 IParser 耦合。

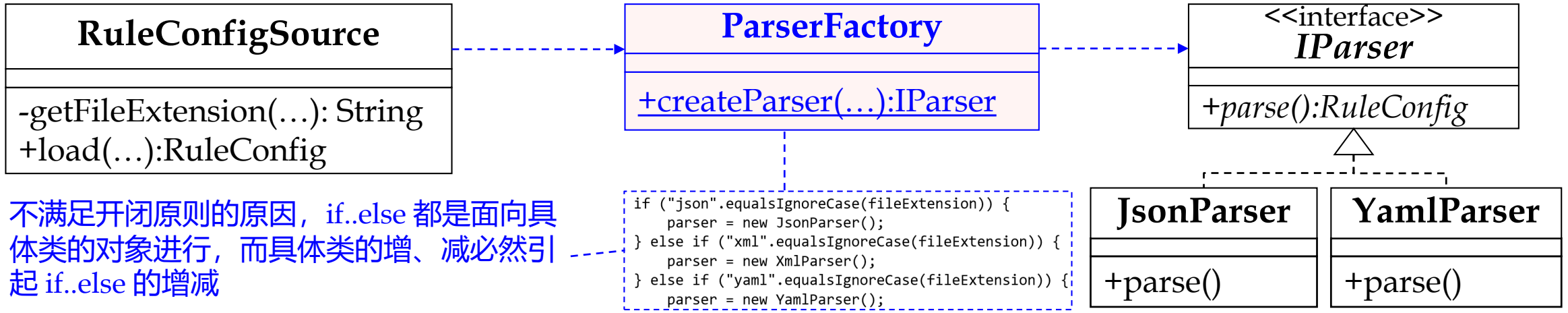
②多客户端时，每个客户端都写一遍，不满足 DRY 原则



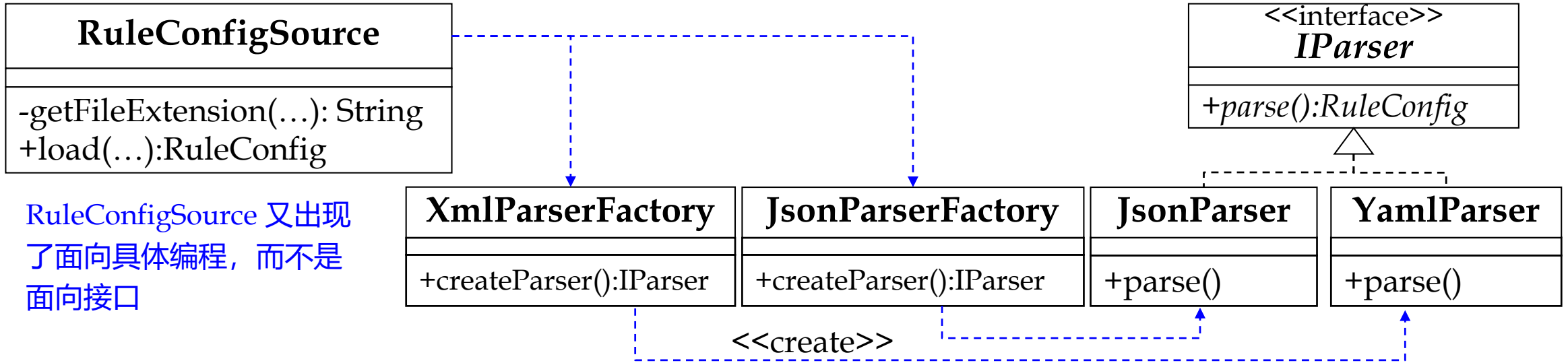
分离出一个工厂类，专门负责产生对象。该工厂类使得客户端与 IParser 解耦。但在工厂类中，if..else 依然存在。

简单工厂 不满足开闭原则 OCP

5.2 进一步职责分离，具体工厂产生具体类

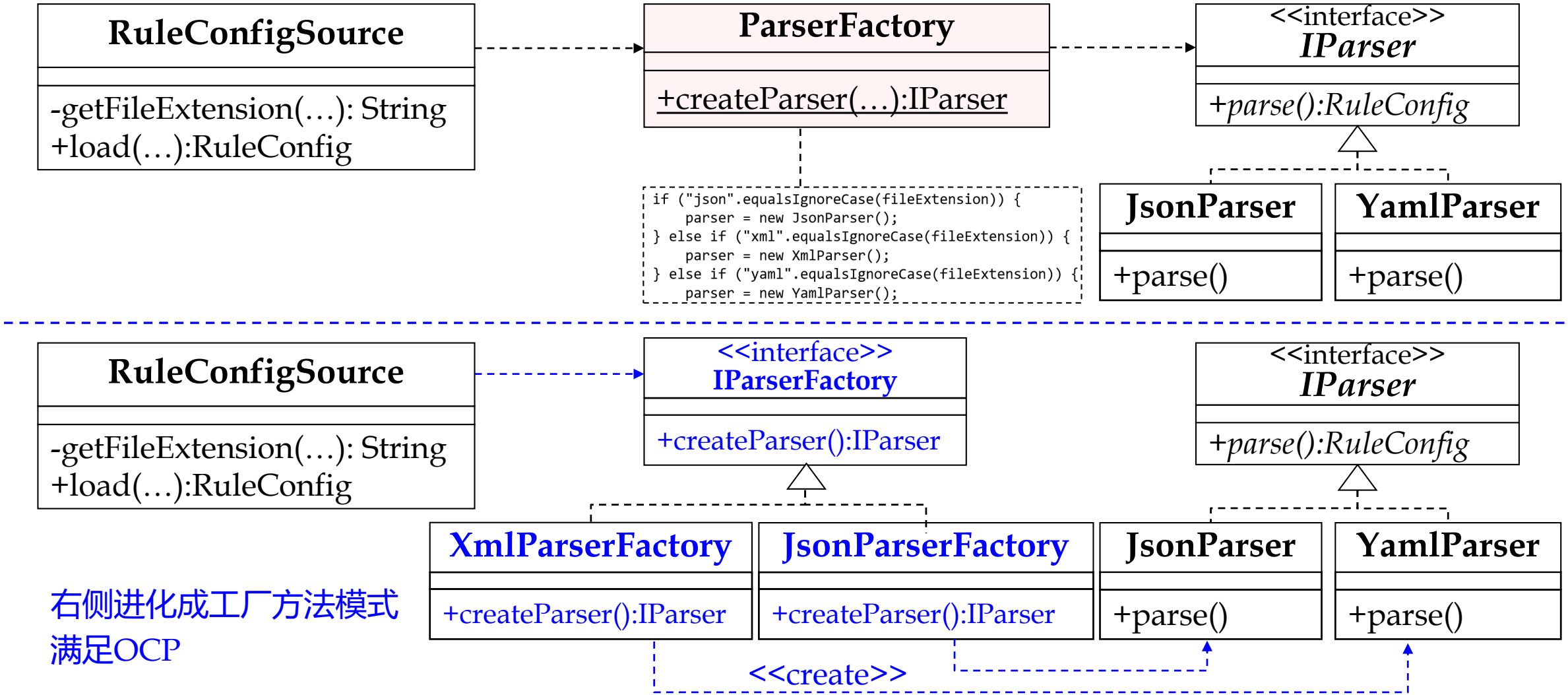


不满足开闭原则的原因，if..else 都是面向具体类的对象进行，而具体类的增、减必然引起 if..else 的增减



RuleConfigSource 又出现了面向具体编程，而不是面向接口

5.3 碰到具体，要抽象-形成工厂的层次类



右侧进化成工厂方法模式
满足OCP

5.4 讨论：简单工厂的解耦问题

```
// 简单工厂的实现方式二-查表法
public class ParserFactory {
    private static final Map<String, IParser> cachedParsers = new HashMap<>(); // 缓存

    static { // 类加载的时候创建
        cachedParsers.put("json", new JsonParser());
        cachedParsers.put("xml", new XmlParser());
        cachedParsers.put("yaml", new YamlParser());
        cachedParsers.put("properties", new PropertiesParser());
    }

    public static IParser createParser(String fileExtension) {
        if (configFormat == null || configFormat.isEmpty()) {
            return null; //返回null还是IllegalArgumentException全凭你自己说了算
         }
        IParser parser = cachedParsers.get(fileExtension.toLowerCase());
        return parser;
    }
}
```

最接近满足 OCP 的实现方式

5.4 讨论：简单工厂的解耦问题

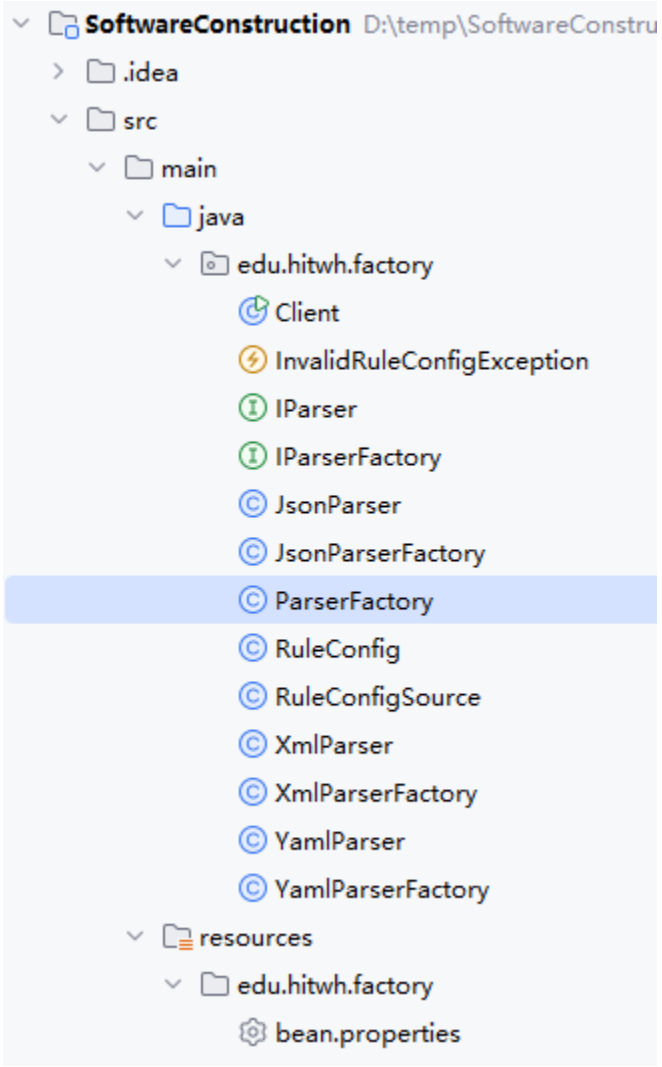
- 如何才能给简单工厂解耦？
- 思路：可以通过简单工厂模式+配置文件的方式解除 工厂对象 和 产品对象 的耦合。在简单工厂类中加载配置文件中的全类名，并创建对象进行存储，客户端如果需要对象，直接进行获取即可
- 在 Java 的 Maven 工程中的 resource 文件夹下创建对应的文件夹，并放置配置文件 productbean.properties
- 格式：配置文件的后缀名=该类型对应产品类的全类名

```
json=edu.hitwh.factory.JsonParser
xml=edu.hitwh.factory.XmlParser
yaml=edu.hitwh.factory.YamlParser
```

5.4 讨论：简单工厂的解耦问题

```
public class ParserFactory {
    private static HashMap<String, IParser> map = new HashMap<>(); // 定义容器
    // 加载配置文件，  只需要加载一次
    static {
        Properties p = new Properties(); // 创建 Properties 对象
        // 加载配置文件
        InputStream is = ParserFactory.class.getClassLoader().getResourceAsStream("product.properties");
        try {
            p.load(is);
            Set<Object> keys = p.keySet(); //从 p 集合中获取全类名并创建对象
            for (Object key : keys) {
                String className = p.getProperty((String) key);
                Class clazz = Class.forName(className); //通过反射技术创建对象
                IParser parser = (IParser) clazz.getDeclaredConstructor().newInstance();
                map.put((String) key, parser); //将名称和对象存储到容器中
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    //根据名称获取对象
    public static IParser getParser(String fileExtension) {
        return map.get(fileExtension);
    }
}
```

5.4 讨论：简单工厂的解耦问题




```java
public class Client {
    public static void main(String[] args) {
        IParser parser = ParserFactory.getParser("json");
        System.out.println(parser);
        parser = ParserFactory.getParser("xml");
        System.out.println(parser);
        parser = ParserFactory.getParser("yaml");
        System.out.println(parser);
    }
}
```

```
D:\tools\jdk-17.0.12\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2
edu.hitwh.factory.JsonParser@6d03e736
edu.hitwh.factory.XmlParser@568db2f2
edu.hitwh.factory.YamlParser@378bf509

Process finished with exit code 0
```

基于此思路，可以解决简单工厂的耦合问题。而且满足 OCP，易扩展

6. 建造者模式
Builder Pattern

6.1 由一道大厂笔试题引发的设计

- 我们需要定义一个资源池配置类 ResourcePoolConfig。这里的资源池，你可以简单理解为线程池、连接池、对象池等。
- 在这个资源池配置类中，有以下几个成员变量，也就是可配置项。现在，请你编写代码实现这个 ResourcePoolConfig 类。

成员变量	解释	是否必填	默认值
name	资源名称	是	没有
maxTotal	最大总资源数量	否	8
maxIdle	最大空闲资源数量	否	8
minIdle	最小空闲资源数量	否	0

6.1 由一道大厂笔试题引发的设计

```
public class ResourcePoolConfig {
    private static final int DEFAULT_MAX_TOTAL = 8;
    private static final int DEFAULT_MAX_IDLE = 8;
    private static final int DEFAULT_MIN_IDLE = 0;
    private String name;
    private int maxTotal = DEFAULT_MAX_TOTAL;
    private int maxIdle = DEFAULT_MAX_IDLE;
    private int minIdle = DEFAULT_MIN_IDLE;

    public ResourcePoolConfig(String name, Integer maxTotal, Integer maxIdle, Integer minIdle) {
        if (StringUtils.isBlank(name)) {
            throw new IllegalArgumentException("name cannot be blank");
        }
        this.name = name;
        if (maxTotal != null) {
            if (maxTotal <= 0) {
                throw new IllegalArgumentException("maxTotal should be positive.");
            }
            this.maxTotal = maxTotal;
        }
        // 其他两个参数也是如上处理，页面限制省略…
        // 省略 getter 方法
    }
}
```

①构造函数的参数列表会变得很长，代码在可读性和易用性上都会变差

②使用构造函数的时候，我们就容易搞错各参数的顺序，传递进错误的参数值，导致非常隐蔽的 bug。

6.2 如何解决构造函数参数长列表问题

```
// 参数太多，导致可读性差、参数可能传递错误
ResourcePoolConfig resourcePoolConfig = new ResourcePoolConfig("datasource", 16, null, 0);
```

- 构造函数中，入参过多，此时就应该避免用构造函数注入参数。还有其他的参数注入方式否?
  - 可以通过 setter 注入
  - 可以通过 Spring 框架 用 @Autowired，@Resource 等自动注入

6.2 如何解决构造函数参数长列表问题-setter注入

```
public class ResourcePoolConfig {
    private static final int DEFAULT_MAX_TOTAL = 8;
    private static final int DEFAULT_MAX_IDLE = 8;
    private static final int DEFAULT_MIN_IDLE = 0;
    private String name;
    private int maxTotal = DEFAULT_MAX_TOTAL;
    private int maxIdle = DEFAULT_MAX_IDLE;
    private int minIdle = DEFAULT_MIN_IDLE;

    public ResourcePoolConfig(String name) {
        if (StringUtils.isBlank(name)) {
            throw new IllegalArgumentException("name cannot be blank");
        }
        this.name = name;
    }
    public void setMaxTotal(int maxTotal) {
        if (maxTotal <= 0) {
            throw new IllegalArgumentException("maxTotal should be positive.");
        }
        this.maxTotal = maxTotal;
    }
    // 其他两个参数也是如上处理，页面限制省略…
    //...省略getter方法...
}
```

6.2 如何解决构造函数参数长列表问题-setter注入

- 新的 ResourcePoolConfig 类的使用举例

成员变量	解释	是否必填	默认值
name	资源名称	是	没有
maxTotal	最大总资源数量	否	16
maxIdle	最大空闲资源数量	否	8
minIdle	最小空闲资源数量	否	0

```
// 新的 ResourcePoolConfig使用举例
ResourcePoolConfig config = new ResourcePoolConfig(“datasource”);
config.setMaxTotal(16);
config.setMaxIdle(8);
```

name 是必填项，用构造函数注入；其他三项选填项，用 setter 进行设置，使用者自主选择填写与否。

没有了冗长的函数调用和参数列表，代码在可读性和易用性上提高了很多

6.3 从架构师视角深入分析，需要考虑的问题

序号	问题	导致的结果
1	必填配置项多。 必填配置项都放到构造函数中设置，那构造函数就又会出现参数列表很长的问题。	把必填项也通过 setter注入，那无法保证所有的参数都set了。
2	配置项之间有依赖关系。 如果用户设置了 maxTotal、maxIdle、minIdle 其中一个，就必须显式地设置另外两个； 或者配置项之间有一定的约束条件，比如，maxIdle 和 minIdle 要小于等于 maxTotal。	刚才的设计思路，配置项之间的依赖关系的校验逻辑就没办法写了。
3	如果要求 ResourcePoolConfig 类对象是不可变对象 也就是说，对象在创建好之后，就不能再修改内部的属性值。	要实现这个功能，ResourcePoolConfig 类就不能暴露 set() 方法， 即：无法再用 setter。参数需要通过构造函数注入。

6.3 从架构师视角深入分析，需要考虑的问题-解决思路

```
// 参数太多，导致可读性差、参数可能传递错误
public ResourcePoolConfig(String name, Integer maxTotal, Integer maxIdle, Integer minIdle)
```

方案：将这些参数封装成一个类，命名为 Builder，此时 ResourcePoolConfig 的构造函数的参数就只有一个了。

```
public ResourcePoolConfig(Builder builder)
```

在Builder中，实现：

①初始化封装的参数：在类中通过 setter 设置参数的值。

②校验参数关系：做集中校验（必填项校验、依赖关系校验、约束条件用builder类中的build方法校验），校验通过之后创建 ResourcePoolConfig 对象

Builder
-name:String -maxTotal:Integer -maxIdle: Integer -minIdle: Integer
+setName():void +setMaxTotal(maxTotal:int):void +setMaxIdle(maxIdle:int):void +setMinIdle(minIdle:int):void +build():ResourcePoolConfig

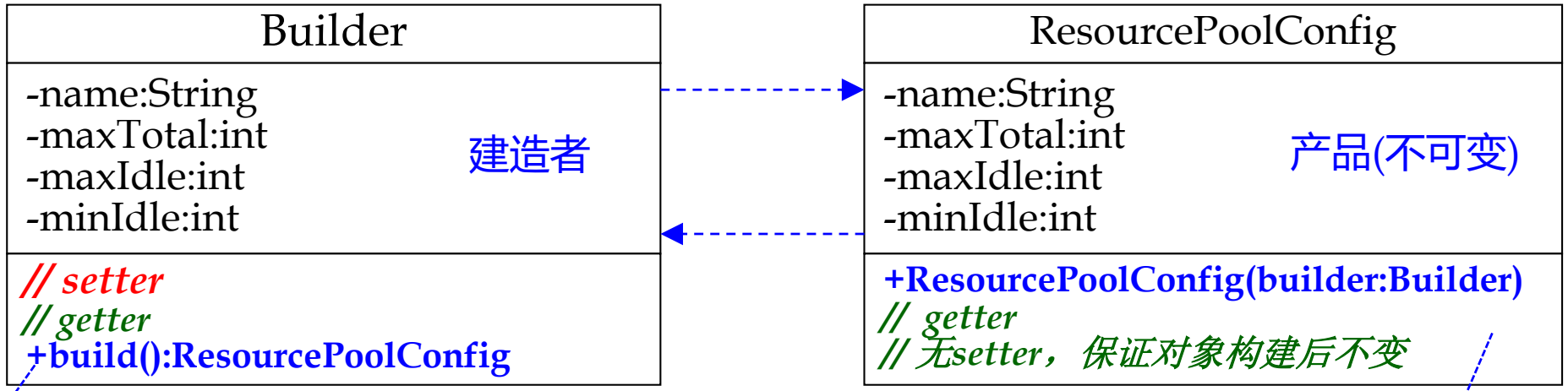
①

```
if (StringUtils.isBlank(name)) {
    throw new IllegalArgumentException("...");
}
// 其他的校验逻辑
…
return new ResourcePoolConfig(this)
```

②

6.3 从架构师视角深入分析，需要考虑的问题-解决思路

既然ResourcePoolConfig 不可变，那就把其入参Builder对象先做好，再传给ResourcePoolConfig




```
public ResourcePoolConfig build() {
    //必填项，依赖，约束条件校验逻辑放到这里来做
    if (StringUtils.isBlank(name)) {
        throw new IllegalArgumentException("...");
    }
    if (maxIdle > maxTotal) {
        throw new IllegalArgumentException("...");
    }
    if (minIdle > maxTotal || minIdle > maxIdle) {
        throw new IllegalArgumentException("...");
    }
    return new ResourcePoolConfig(this);
}
```

```
public ResourcePoolConfig(Builder builder) {
    this.name = builder.getName();
    this.maxIdle = builder.getMaxIdle();
    this.minIdle = builder.getMinIdle();
    this.maxTotal = builder.getMaxTotal();
}
```

①校验逻辑放置到 Builder 类中。先创建 Builder 对象 builder，并且通过 setter 设置 builder 中的变量值，②然后，在使用 builder 的 build() 方法真正创建对象之前，做集中的校验；③校验通过之后builder才会创建 ResourcePoolConfig 对象。

6.3 从架构师视角深入分析，需要考虑的问题-传统实现

```
// 产品类，用Builder封装了参数
public class ResourcePoolConfig {
    private static final int DEFAULT_MAX_TOTAL = 8;
    private static final int DEFAULT_MAX_IDLE = 8;
    private static final int DEFAULT_MIN_IDLE = 0;
    private String name;
    private int maxTotal = DEFAULT_MAX_TOTAL;
    private int maxIdle = DEFAULT_MAX_IDLE;
    private int minIdle = DEFAULT_MIN_IDLE;

    public ResourcePoolConfig(Builder builder) {
        this.name = builder.getName();
        this.maxIdle = builder.getMaxIdle();
        this.minIdle = builder.getMinIdle();
        this.maxTotal = builder.getMaxTotal();
    }
    // getter 略…
    // 无setter，因为根据需求，该类不可变
}
```

6.3 从架构师视角深入分析，需要考虑的问题-传统实现

```
public class Builder {
    private static final int DEFAULT_MAX_TOTAL = 8;
    private static final int DEFAULT_MAX_IDLE = 8;
    private static final int DEFAULT_MIN_IDLE = 0;
    private String name;
    private int maxTotal = DEFAULT_MAX_TOTAL;
    private int maxIdle = DEFAULT_MAX_IDLE;
    private int minIdle = DEFAULT_MIN_IDLE;

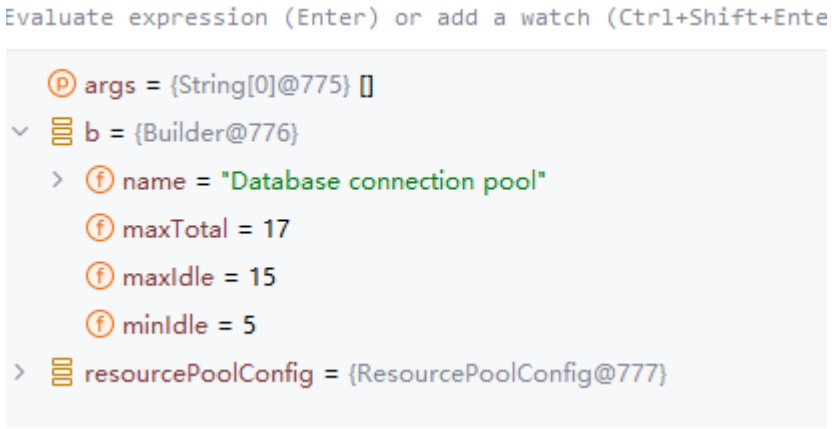
    public void setName(String name) {
        if (StringUtils.isBlank(name)) {
            throw new IllegalArgumentException("...");
        }
        this.name = name;
    }
    public void setMaxTotal(int maxTotal) {
        if (maxTotal <= 0) {
            throw new IllegalArgumentException("...");
        }
        this.maxTotal = maxTotal;
    }   // 转下一页
```

6.3 从架构师视角深入分析，需要考虑的问题-传统实现

```
    // 接上一页
    public void setMinIdle(int minIdle) {
        if (minIdle < 0) {
            throw new IllegalArgumentException("...");
        }
        this.minIdle = minIdle;
    }
    public ResourcePoolConfig build() {
        // 校验逻辑放到这里来做，包括必填项校验、依赖关系校验、约束条件校验等
         if (StringUtils.isBlank(name)) {
            throw new IllegalArgumentException("...");
        }
        if (maxIdle > maxTotal) {
            throw new IllegalArgumentException("...");
        }
        if (minIdle > maxTotal || minIdle > maxIdle) {
            throw new IllegalArgumentException("...");
        }
        return new ResourcePoolConfig(this);
    }
}
```

6.3 从架构师视角深入分析，需要考虑的问题-传统实现

```java
public class Client {
    public static void main(String[] args) {
        Builder b = new Builder();
        b.setName("Database connection pool");
        b.setMaxTotal(17);
        b.setMaxIdle(15);
        b.setMinIdle(5);
        ResourcePoolConfig resourcePoolConfig = b.build();
        System.out.println(resourcePoolConfig.toString());
    }
}
```




```
D:\tools\jdk-17.0.12\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.4.1\lib\idea_rt.jar=1419" -Dfile
builder.ResourcePoolConfig@682a0b20

Process finished with exit code 0
```

6.3 从架构师视角深入分析，需要考虑的问题-传统实现

```java
public class Client {
    public static void main(String[] args) {
        Builder b = new Builder();
        b.setName("Database connection pool");
        b.setMaxTotal(17);
        b.setMaxIdle(15);
        b.setMinIdle(5);
        ResourcePoolConfig resourcePoolConfig = b.build();
        System.out.println(resourcePoolConfig.toString());
    }
}
```

通过多个方法调用，逐步配置对象属性，而非一次性传入所有参数

构建与表示分离：
通过封装构造逻辑到独立的 Builder 类中，客户端无需关心具体构建细节（即build方法的逻辑）

```
D:\tools\jdk-17.0.12\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.3.4.1\lib\idea_rt.jar=1419" -Dfile
builder.ResourcePoolConfig@682a0b20

Process finished with exit code 0
```

6.3 从架构师视角深入分析，需要考虑的问题-链式编程

```
public class Client {
    public static void main(String[] args) {
        Builder b = new Builder();
        b.setName("Database connection pool");
        b.setMaxTotal(17);
        b.setMaxIdle(15);
        b.setMinIdle(5);
        ResourcePoolConfig resourcePoolConfig = b.build();
        System.out.println(resourcePoolConfig.toString());
    }
}
```

传统的参数设置方式，可读性稍微差一点。

```
public class Client {
    public static void main(String[] args) {
        ResourcePoolConfig resourcePoolConfig = new Builder()
                .setName("Database connection pool")
                .setMaxTotal(17)
                .setMaxIdle(15)
                .setMinIdle(5)
                .build();
        System.out.println(resourcePoolConfig.toString());
    }
}
```

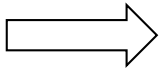
链式编程。可读性好一点，且能够体现逐步设置参数，设置完毕后再构建的思想

当然，链式编程并非构建者模式必须方式。

6.3 从架构师视角深入分析，需要考虑的问题-链式编程

传统编程的设置参数并构造产品对象的设计

Builder
-name:String -maxTotal:Integer -maxIdle: Integer -minIdle: Integer
// 省略 getter +setName():void +setMaxTotal(maxTotal:int):void +setMaxIdle(maxIdle:int):void +setMinIdle(minIdle:int):void +build():ResourcePoolConfig



链式编程的设置参数并构造产品对象的设计

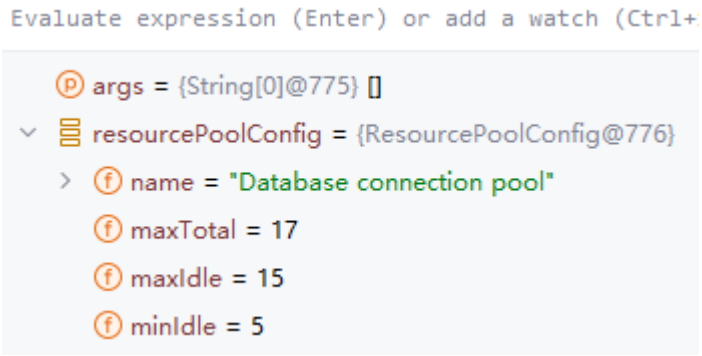
Builder
-name:String -maxTotal:Integer -maxIdle: Integer -minIdle: Integer
// 省略 getter +setName():**Builder** +setMaxTotal(maxTotal:int):**Builder** +setMaxIdle(maxIdle:int):**Builder** +setMinIdle(minIdle:int):**Builder** +build():ResourcePoolConfig

6.3 从架构师视角深入分析，需要考虑的问题-链式编程

```
public class Builder {
    //省略常量定义，与之前的代码相同
    private String name;
    private int maxTotal = DEFAULT_MAX_TOTAL;
    private int maxIdle = DEFAULT_MAX_IDLE;
    private int minIdle = DEFAULT_MIN_IDLE;
    //省略 getters
    public Builder setName(String name) { // 原来是 void
        if (StringUtils.isBlank(name)) {
            throw new IllegalArgumentException(“...”);
        }
        this.name = name;
        return this;  // 增加此句，表示返回对象自身。
    }
    public Builder setMaxTotal(int maxTotal) {
        if (maxTotal <= 0) {
            throw new IllegalArgumentException("...");
        }
        this.maxTotal = maxTotal;
        return this;
    }
    // 其他参数设置同上；build() 方法代码无变化，略…
}
```

6.3 从架构师视角深入分析，需要考虑的问题-链式编程

```
public class Client {
    public static void main(String[] args) {
        ResourcePoolConfig resourcePoolConfig = new Builder()
                .setName("Database connection pool")
                .setMaxTotal(17)
                .setMaxIdle(15)
                .setMinIdle(5)
                .build();
        System.out.println(resourcePoolConfig.toString());
    }
}
```



另一种更常见的写法：

```
ResourcePoolConfig resourcePoolConfig = new ResourcePoolConfig.Builder()
                .setName("Database connection pool")
                .setMaxTotal(17)
                .build();
```

这种写法，显然是将 Builder 类，写在 ResourcePoolConfig 类中，作为内部静态类。

此时 ResourcePoolConfig 的构造函数可以设置为私有，
只能通过静态类的 build() 方法产生 ResourcePoolConfig 对象。代码如下：

6.3 从架构师视角深入分析，需要考虑的问题-静态内部类

```
public class ResourcePoolConfig {
    private String name; //去掉常量默认值
    private int maxTotal;
    private int maxIdle;
    private int minIdle;

    private ResourcePoolConfig(Builder builder) {
        this.name = builder.name;
        this.maxIdle = builder.maxIdle;
        this.minIdle = builder.minIdle;
        this.maxTotal = builder.maxTotal;
    }

    public static class Builder {
        //与之前链式编程的代码完全一致
    }
}
```

上层模块调用：

```
ResourcePoolConfig resourcePoolConfig = new ResourcePoolConfig.Builder() // 调用无参构造，产生对象
                .setName("Database connection pool")   // 链式编程进行参数分布设定，最后调用 build()
                //… 其他参数设定及build() 省略。
```

6.4 建造者模式 Builder-定义

- 建造者模式（Builder Pattern），也叫做生成器模式，其定义如下：

Separate the construction of a complex object from its representation so that the same construction process can create different representations.
将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。

什么是表示 representations？

“表示” 在此处理解为具体成品的模样。燃油车表示 = 油箱 + 汽油发动机；电动车表示 = 电池组 + 电动机。相同的组装流程（建造过程），但最终产品形态/模样完全不同

将一个复杂对象的构建与它的表示分离，又是啥意思？

你想要一台特斯拉Model Y （表示），这台车在流水线如何组装组装的不必关心（构建）。表示和构建分离，能灵活定制，隐藏复杂度

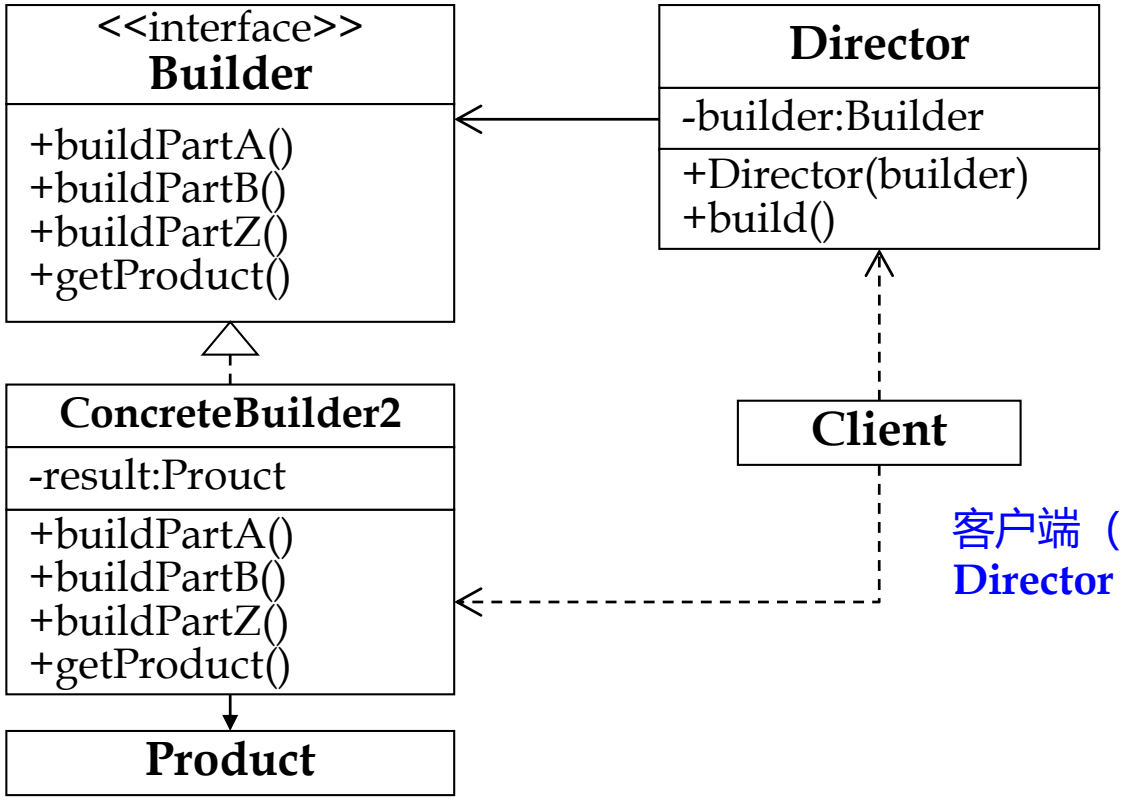
使得同样的构建过程可以创建不同的表示

“表示”=你手里拿到的最终产品，而“建造过程”=背后组装它的流水线的运行过程。流水线固定不变（建造者），但通过换零件，同一流水线能产出跑车、卡车、房车（不同表示）

6.4 建造者模式 Builder-结构

建造者/生成器（Builder）接口声明在所有类型生成器中通用的产品构造步骤。知道单独的步骤。但这些步骤的顺序建造者不知道

主管（Director）类定义调用构造步骤的顺序，这样你就可以创建和复用特定的产品配置。



具体生成器（Concrete Builders）提供构造过程的不同实现。既然是子类，显然可以增加不同的建造器

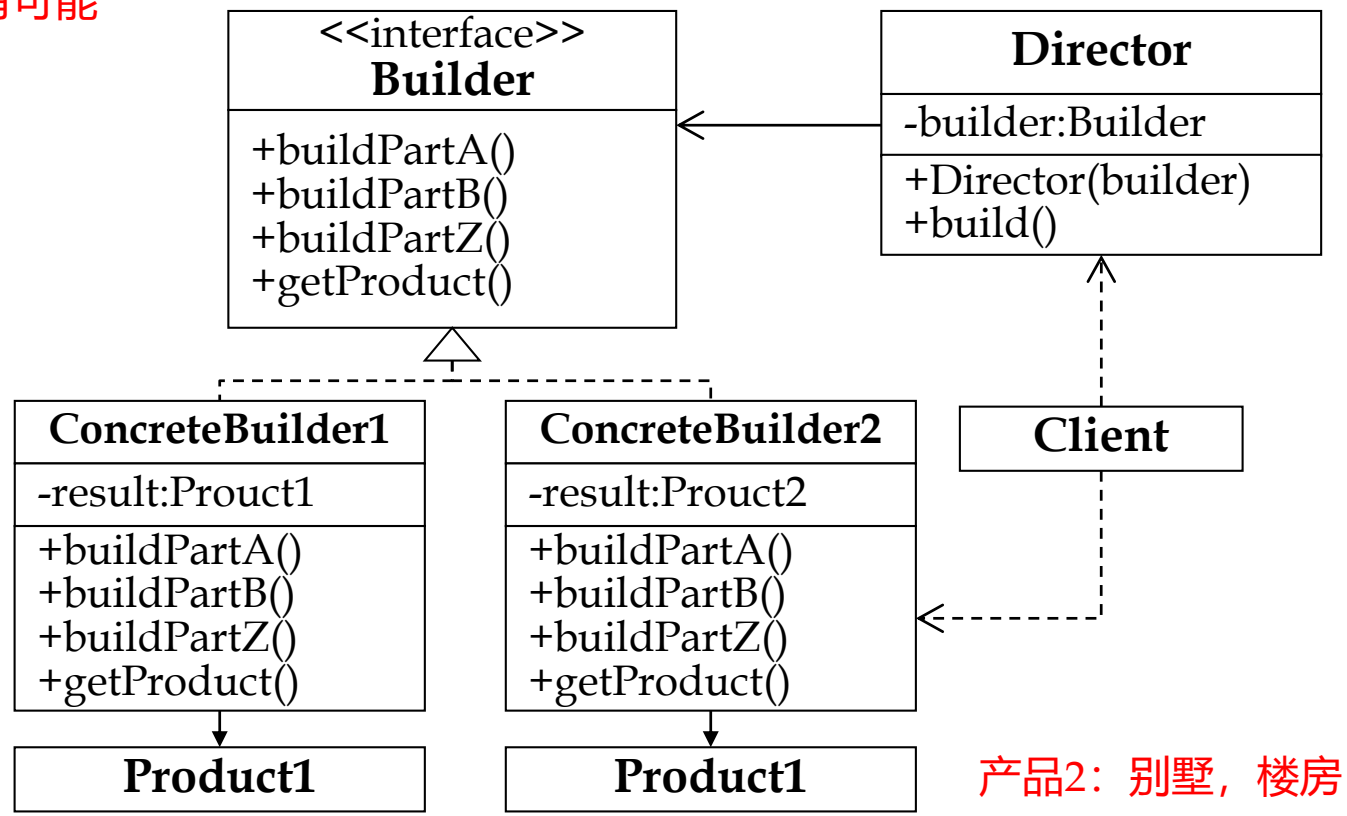
客户端（Client）需要知道 **Director** 和具体的建造者

产品（Products）是最终生成的对象。同一具体建造器可以建造同一类型不同形式的产品（例如石头或木头房屋）。

6.4 建造者模式 Builder-结构

基本生成器接口中定义了所有可能的制造步骤。

主管类将负责管理制造步骤的顺序。

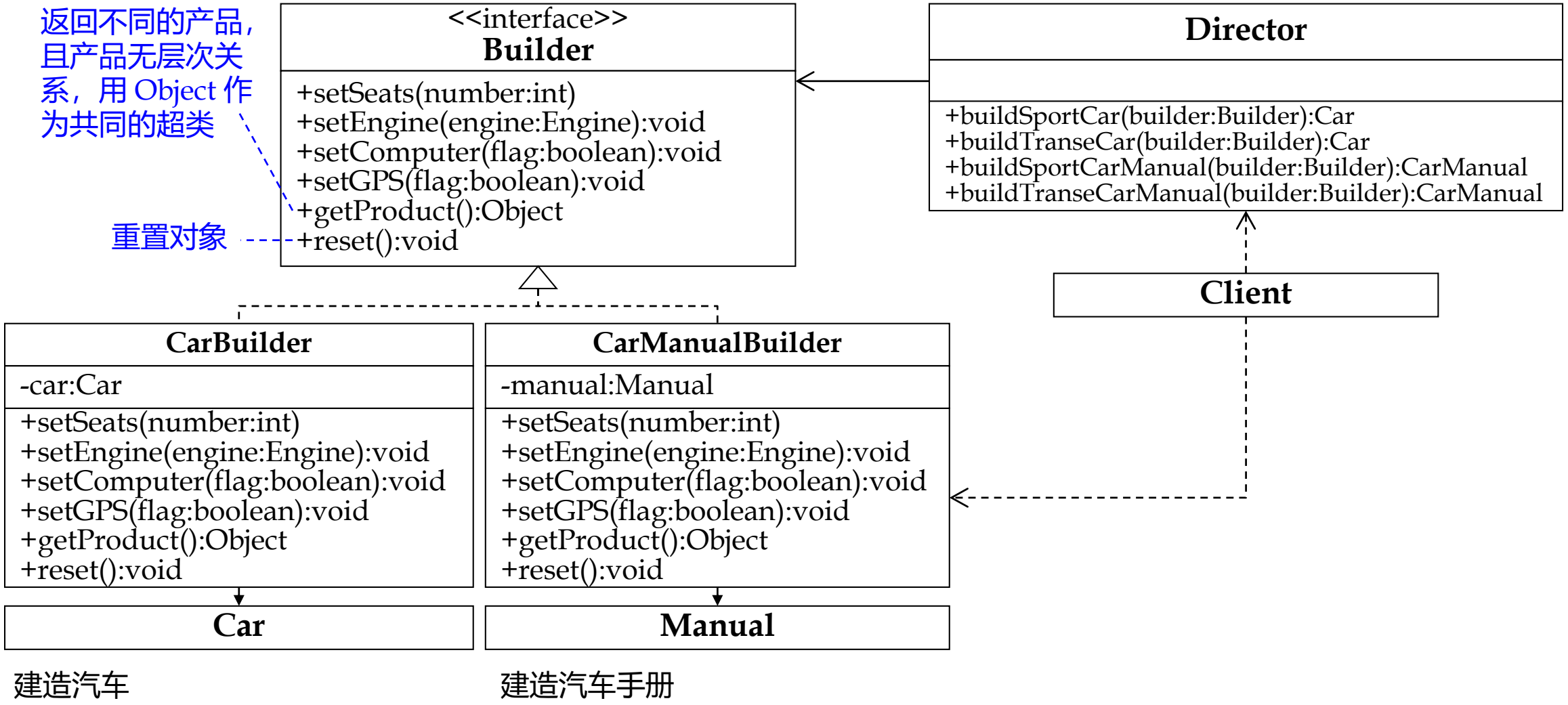


产品1：小汽车，卡车

产品2：别墅，楼房

可以由不同生成器构造的不同类型的产品（制造过程相似且仅有细节上的差异），且无需属于同一类层次结构或接口。

6.5 练习-分步骤制造汽车及对应用户手册



6.5 练习-分步骤制造汽车及对应用户手册

```
// 生成器接口声明了创建产品对象不同部件的方法
public interface Builder {
    void setSeats(int number);
    void setEngine(Engine engine);
    void setComputer(boolean flag);
    void setGPS(boolean flag);
    // 通常在生成器实例将结果返回给客户端后，它们应该做好生成另一个产品的准备。
   // 因此生成器实例通常会在 `getProduct（获取产品）`方法主体末尾调用重置方法。

    Object getProduct();
    void reSet();
}
```

6.5 练习-分步骤制造汽车及对应用户手册

```
public class CarBuilder implements Builder {
    private Car car ;
    public CarBuilder() {
        this.reSet();
    }
    @Override
    public void setSeats(int number) {
        // 设置汽车座位的数量。
     }
    // 省略部分代码….

    @Override
    public Object getProduct() {
        return this.car;
    }
    @Override
    public void reSet() {
        this.car = new Car();
    }
```

6.5 练习-分步骤制造汽车及对应用户手册

```
public class CarManualBuilder implements Builder{
    private CarManual carManual;
    public CarManualBuilder() {
        this.reSet();
    }
    @Override
    public void setSeats(int number) {
        // 添加关于汽车座椅功能的文档。
     }
    // 其他代码省略
    @Override
    public Object getProduct() {
        return this.carManual;
    }
    @Override
    public void reSet() {
        this.carManual = new CarManual();
    }
}
```

6.5 练习-分步骤制造汽车及对应用户手册

```
public class Director {
    // 创建汽车说明书
    public Car buildSportCar(Builder builder) {
        builder.reSet();
        builder.setSeats(5);
        builder.setEngine(new SportEngine());
        builder.setComputer(true);
        builder.setGPS(true);
        return (Car)builder.getProduct();
    }
    // 创建汽车说明书
    public CarManual buildSportCarManual(Builder builder) {
        builder.reSet();
        builder.setSeats(5);
        builder.setEngine(new SportEngine());
        builder.setComputer(true);
        builder.setGPS(true);
        return (CarManual) builder.getProduct();
    }
}
```



Builder中的方法，只是一个个部分的构造，并没有构建顺序。Director 中的构建方法，限制了构造的顺序

6.5 练习-分步骤制造汽车及对应用户手册

```
public class Client {
    public static void main(String[] args) {

        Director director = new Director();
        Builder carBuilder = new CarBuilder();
        Car sportCar = director.buildSportCar(carBuilder);
        System.out.println(sportCar);

        Builder carManualBuilder = new CarManualBuilder();
        CarManual sportCarManual = director.buildSportCarManual(carManualBuilder);
        System.out.println(sportCarManual);
    }
}
```

```
D:\tools\jdk-17.0.9\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.1.1.1\lib\idea_rt.jar=13786" -
edu.hitwh.builder.Car@448139f0
edu.hitwh.builder.CarManual@3b9a45b3

Process finished with exit code 0
```

练习3：重构建造电脑构造过程

- 电脑主机的主要部件包括cpu，主板，内存，硬盘等。不同的电脑发烧友对部件的要求不一样，如何根据不同的元器件，产生不同的电脑主机对象？







练习3：重构建造电脑构造过程

```
public class PC {
    private String cpu;
    private String memory;
    private String mainboard;
    private String power;
    private String disk;
    private String graphics;

    public PC(String cpu, String memory, String mainboard,
                          String power, String disk, String graphics) {
        this.cpu = cpu;
        this.memory = memory;
        this.mainboard = mainboard;
        this.power = power;
        this.disk = disk;
        this.graphics = graphics;
    }
    // setter和getter略...
}
```

练习3：重构建造电脑构造过程

```
public class Client {
    public static void main(String[] args) {
        //构建CompatiblePC对象
        PC pc = new PC("intel i11",
                       "三星32G",
                       "华硕",
                       "华硕",
                       "西部数据",
                       "Geforece3900");

        System.out.println(pc);
    }
}
```

代码可读性会非常差，而且很容易引入错误。

- 要求：解决代码可读性会非常差，很容易引入错误的问题

本章总结

序号	设计模式	一句话归纳	目的
1	单例模式 Singleton Pattern	一个类只允许创建一个对象	在牺牲部分灵活性的前提下，通过严格的实例控制解决资源共享与一致性难题
2	简单工厂 Simple Factory Pattern	封装对象的创建逻辑，降低客户端代码与具体产品类之间的耦合度	屏蔽内部创建细节，提高扩展灵活性，统一创建接口
3	工厂方法模式 Factory Method Pattern	将对象的创建过程延迟到子类，让子类决定实例化哪个具体类	解耦对象的创建逻辑与使用逻辑，提升代码的灵活性和扩展性
4	抽象工厂 Abstract Factory Pattern	通过统一的接口创建一组相互关联或依赖的产品对象，隐藏具体类的实例化逻辑	一个工厂负责创建多个不同类型的对象，有效地减少工厂类的个数
5	建造者模式 Builder Pattern	将复杂对象的构建过程与其表示形式分离，使相同的构建逻辑能生成不同结构的对象	解耦构建逻辑与对象结构，简化复杂对象构造，提升扩展性
~~6~~	~~原型模式~~ ~~Prototype Pattern~~	~~复制现有对象来创建新对象~~	~~避免复杂的初始化过程，提高对象创建效率~~

The end

November 27, 2025

```
public class PC {
    private String cpu;
    private String memory;
    private String mainborad;
    private String power;
    private String disk;
    private String graphics;

    private PC(Builder builder) {
        this.cpu = builder.cpu;
        this.memory = builder.memory;
        this.mainborad = builder.mainborad;
        this.power = builder.power;
        this.disk = builder.disk;
        this.graphics = builder.graphics;
    }
    @Override
    public String toString() {
        return "Builder{" +
                "cpu='" + cpu + '\'' +
                ", memory='" + memory + '\'' +
                ", mainborad='" + mainborad + '\'' +
                ", power='" + power + '\'' +
                ", disk='" + disk + '\'' +
                ", graphics='" + graphics + '\'' +
               '}';
}
// 右侧继续
```

```
// 接左侧
public static final class Builder {

    private String cpu;
    private String memory;
    private String mainborad;
    private String power;
    private String disk;
    private String graphics;

    public Builder cpu(String cpu) {
        this.cpu = cpu;
        return this;
    }

    public Builder memory(String memory) {
        this.memory = memory;
        return this;
    }

   //省略部分代码。结构同上

    public PC build() {
        return new PC(this);
    }
}
```

```java
public class Client {
    public static void main(String[] args) {
        PC pc = new PC.Builder()
                .cpu("intel i11")
                .mainborad("华硕")
                .memory("三星32G")
                .disk("西数2T")
                .power("长城P400")
                .graphics("Geforece3900")
                .build();
        System.out.println(pc);
    }
}
```

```
D:\tools\jdk-17.0.9\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.1\lib\idea_rt.jar=3310" -Dfile.encoding=U
Builder{cpu='intel i11', memory='三星32G', mainborad='华硕', power='长城P400', disk='西数2T', graphics='Geforece3900'}
```